



北方民族大学

本科毕业论文（设计）

题目：基于 Vulkan 的实时光线追踪混合渲染器
的设计与实现

学院名称：_____计算机科学与工程学院_____

专 业：_____软件工程_____

学 生 姓 名：_____向晨_____

学 号：_____20221889_____

指导教师姓名：_____韩强_____

企业指导教师：_____

论文提交时间：_____

北方民族大学本科毕业论文（设计）诚信承诺书

学生姓名	向晨	年级	2022 级
所学专业	软件工程	学号	20221889
所在学院	计算机科学与工程学院		

学生承诺

本人郑重承诺和声明：

我承诺在毕业论文（设计）过程中严格遵守学校有关规定，在指导教师的安排与指导下独立完成所规定的毕业论文（设计）工作，决不弄虚作假，不请别人代做毕业论文（设计）或抄袭别人的成果。所撰写的毕业论文或毕业设计是在指导老师的指导下自主完成，文中所有引文或引用数据、图表均注解并说明来源，本人愿意为由此引起的后果承担责任。

学生（签名）：_____

2026 年 05 月 日

基于 Vulkan 的实时光线追踪混合渲染器的设计与实现

摘要

在现代实时计算机图形学中，传统的纯光栅化渲染在处理全局光照、精确物理软阴影与复杂镜面反射时存在固有的物理局限性，而离线级别的纯路径追踪又难以满足交互式应用的实时帧率需求。针对这一“画质与算力”的矛盾，本文基于现代显式图形 API Vulkan 1.3，设计并实现了一套工业级的高性能实时光线追踪混合渲染引擎。

本文的核心工作包括三个方面。首先，针对 Vulkan 繁琐的显存状态同步痛点，设计了数据驱动的 Render Graph 调度架构，通过有向无环图的拓扑分析实现了管线屏障的自动推导与显存复用优化。其次，构建了高效的混合光照管线，将几何光栅化与硬件光线追踪深度解耦。在延迟渲染 G-Buffer 的基础上，结合偏导数面法线偏移与 Ray Query 扩展实现了精准无伪影的软硬阴影，并利用 RT Pipeline 全面求解 Cook-Torrance 微面元模型，完成了基于物理（PBR）的多次镜面反射计算。最后，针对 1 SPP 极低采样率带来的蒙特卡洛高频噪声，完整集成了时空方差导向滤波（SVGF）降噪系统。通过几何运动矢量的时域历史重投影与动态方差引导的多轮 A-Trous 空域滤波，成功实现了高保真度的抗噪平滑处理。

性能测试与效果分析表明，本系统在 2K 原生分辨率下，能够以实时的性能开销（稳定大于 60 帧）输出平滑且物理正确的照片级渲染画面。本文的研究验证了以 Render Graph 为底座、结合光栅化、硬件光追与 SVGF 降噪的混合架构在现代底层图形引擎开发中的高效性与工程可行性。

关键词: Vulkan; 实时光线追踪; 混合渲染; Render Graph; 基于物理的渲染 (PBR); SVGF 降噪

Design and Implementation of a Real-time Ray Tracing Hybrid Renderer Based on Vulkan

ABSTRACT

In modern real-time computer graphics, traditional pure rasterization rendering has inherent physical limitations when dealing with global illumination, precise physical soft shadows, and complex specular reflections, while offline-level pure path tracing is difficult to meet the real-time frame rate requirements of interactive applications. To address this contradiction between "image quality and computing power", this paper designs and implements an industrial-grade high-performance real-time ray tracing hybrid rendering engine based on the modern explicit graphics API Vulkan 1.3.

The core work of this paper includes three aspects. First, a data-driven Render Graph scheduling architecture was designed to address the tedious GPU memory state synchronization pain points of Vulkan. Through topological analysis of a directed acyclic graph, the automatic derivation of pipeline barriers and optimization of memory reuse were achieved. Second, an efficient hybrid lighting pipeline was constructed to deeply decouple geometric rasterization from hardware ray tracing. Based on deferred rendering G-Buffer, combined with derivative face normal offset and Ray Query extension, precise and artifact-free soft and hard shadows were implemented. Furthermore, the RT Pipeline was used to comprehensively solve the Cook-Torrance microfacet model, completing multi-bounce specular reflection calculations based on physics (PBR). Finally, addressing the high-frequency Monte Carlo noise caused by the extremely low sampling rate of 1 SPP, a complete Spatiotemporal Variance-Guided Filtering (SVGF) denoising system was integrated. High-fidelity anti-noise smoothing was successfully achieved through temporal history reprojection of geometric motion vectors and multi-round spatial A-Trous filtering guided by dynamic variance.

Performance tests and effectiveness analysis show that this system can output smooth and physically correct photo-realistic rendering results with real-time performance overhead (stable above 60 FPS) at 2K native resolution. This research verifies the efficiency and engineering

feasibility of a hybrid architecture based on Render Graph, combining rasterization, hardware ray tracing, and SVGF denoising in modern low-level graphics engine development.

Key words: Vulkan; Real-time Ray Tracing; Hybrid Rendering; Render Graph; Physically Based Rendering (PBR); SVGF Denoising

目 录

摘要	II
ABSTRACT	III
第一章 绪论	1
1.1 研究背景及意义	1
1.2 研究现状	1
1.3 本文研究内容	2
1.4 论文总体结构	2
1.5 本章小结	3
第二章 实时混合渲染相关理论	4
2.1 实时渲染相关基础理论	4
2.1.1 渲染模型	4
2.1.2 渲染器职责与架构设计哲学	4
2.2 图形光栅化原理	4
2.2.1 光栅化渲染管线流程	4
2.2.2 重心坐标插值与透视校正	5
2.2.3 硬件加速与 Early-Z 优化	5
2.3 光线追踪原理	5
2.3.1 光线投射与递归追踪	5
2.3.2 光线追踪的计算挑战	5
2.4 实时混合渲染管线	6
2.4.1 分而治之的渲染策略	6
2.4.2 管线集成与延迟渲染架构	6
2.5 图形 API 与渲染引擎	6
2.5.1 显式图形 API 的技术特征	6
2.5.2 渲染引擎的抽象层职责	6
第三章 需求分析	8
3.1 可行性分析	8

3.1.1	经济可行性	8
3.1.2	技术可行性	8
3.1.3	操作可行性	8
3.2	系统业务流程分析	8
3.3	系统功能性需求分析	9
3.3.1	用户功能需求	9
3.3.1.1	场景与资产管理模块	9
3.3.1.2	Debug UI 界面	10
3.3.1.3	场景交互与观察模块	10
3.4	系统非功能性需求分析	11
3.4.1	性能与响应时间	11
3.4.2	安全性与稳定性	12
3.4.3	可用性与可靠性	12
3.4.4	易用性	12
3.4.5	可维护性与可扩展性	12
3.5	系统核心类设计	13
3.5.1	全局管理与框架类设计	13
3.5.2	资源管理与场景组织类设计	13
3.5.3	渲染调度引擎核心类设计	14
3.6	本章小结	15
第四章	概要设计	16
4.1	渲染器设计概述	16
4.2	系统整体架构设计	16
4.2.1	架构总体思路	16
4.2.2	五层分层拓扑架构	16
4.3	核心功能模块划分	17
4.3.1	核心框架模块	17
4.3.2	资源管理模块	17
4.3.3	场景管理与交互控制模块	18
4.3.4	渲染调度引擎模块	18
4.3.5	混合渲染管线模块	18
4.3.6	可视化与交互调试模块	18
4.4	显存逻辑结构设计	18
4.5	渲染数据流图 (DFD)	19
4.6	本章小结	20

第五章 详细设计	21
5.1 核心框架模块	21
5.1.1 Vulkan 环境初始化与硬件抽象	21
5.1.2 Application 生命周期状态机	21
5.1.2.1 LayerStack 层级管理逻辑	21
5.1.2.2 基于“从顶到底”的事件拦截机制	22
5.1.2.3 输入轮询与连续交互逻辑	22
5.1.2.4 主循环逻辑详细设计	22
5.2 资源管理模块	23
5.2.1 多线程任务系统与异步资源流水线	23
5.2.1.1 TaskSystem 线程池调度逻辑	23
5.2.1.2 后台异步资源处理流水线	24
5.2.2 ResourceManager 与延迟销毁逻辑	25
5.2.2.1 资源对象抽象与 RAII 封装	25
5.2.2.2 基于帧序号的延迟销毁队列 (Deletion Queue)	25
5.2.3 VMA 池化管理与 Bindless 架构设计	25
5.2.3.1 基于 VMA 的显存子分配逻辑	25
5.2.3.2 基于 Bindless 思想的全局描述符绑定	26
5.3 场景管理与交互控制模块	26
5.3.1 场景组织与空间加速算法设计	26
5.3.1.1 基于组合模式的场景实体结构	26
5.3.1.2 视锥体剔除与动态八叉树同步	26
5.3.1.3 硬件加速的双层 BVH 维护	27
5.3.2 EditorCamera 交互算法与坐标变换	27
5.3.2.1 摄像机原理：操作倒转与观察空间	27
5.3.2.2 透视投影与裁剪空间	27
5.3.2.3 MVP 变换流水线推导	28
5.3.2.4 弧球 (Arcball) 相机交互实现	28
5.3.2.5 基于 Reversed-Z 的深度精度优化	28
5.4 渲染调度引擎 (RenderGraph) 详细设计	28
5.4.1 渲染图核心原理与架构设计动机	29
5.4.1.1 自动化流水线工厂类比	29
5.4.1.2 架构设计动机：规避“同步地狱”	29
5.4.2 核心组件设计与架构模式	29
5.4.2.1 指令录制优化：基于静态多态的智能分发	30

5.4.3	运行生命周期与自动化调度逻辑	30
5.4.3.1	Setup: 虚拟依赖构建	31
5.4.3.2	Compile: 拓扑分析与显存优化	31
5.4.3.3	Execute: 自动同步引擎	32
5.4.4	基于微面元理论的 PBR 物理光照详细设计	32
5.4.4.1	双 Lobe 材质评估模型	32
5.4.4.2	法线分布函数与几何遮蔽	33
5.4.4.3	Kulla-Conty 多重散射能量补偿	33
5.4.5	混合渲染管线与信号重建算法详细设计	33
5.4.5.1	SVGF 时域累积与矩估计逻辑	35
5.4.5.1.1	1. 鲁棒重投影校验 (Reprojection Validation)	35
5.4.5.1.2	2. 亮度矩的时域累积	35
5.4.5.2	基于 A-Trous 的多级边缘启发式滤波	35
5.4.5.2.1	1. 统一边缘停止函数 (Edge-Stopping Functions)	35
5.4.5.2.2	2. 方差的空间传播	36
5.4.5.3	光栅化中的离散采样与锯齿现象	36
5.4.5.4	蒙特卡罗积分与重要性采样逻辑 (CDF)	36
5.4.5.4.1	1. 蒙特卡罗估计与方差痛点	36
5.4.5.4.2	2. 重要性采样与 CDF 的推导	37
5.4.5.4.3	3. 硬件加速下的 NEE 策略	37
5.4.5.5	实时光线追踪信号探测与采样算法设计	37
5.4.5.5.1	1. 实时光追软阴影探测 (Ray Traced Shadows)	38
5.4.5.5.2	2. 光线追踪环境光遮蔽 (Ray Traced AO)	38
5.4.5.5.3	3. 1-Bounce 漫反射间接光 (Ray Traced GI)	38
5.4.6	SVGF 时空方差引导降噪算法实现	39
5.4.6.1	反照率解耦 (Albedo Demodulation)	41
5.4.6.2	时域滤波与方差估计 (Temporal Filtering & Variance)	41
5.4.6.3	空间滤波: 联合双边 A-Trous 小波变换	42
5.4.7	时域抗锯齿 (TAA) 方案实现	44
5.4.7.1	TAA 基本原理	44
5.4.7.2	亚像素抖动与采样对齐 (Sub-pixel Jitter)	44
5.4.7.3	速度扩张与最近深度搜索 (Velocity Dilation)	44
5.4.7.4	基于射线相交的历史裁剪 (Ray-Box Clipping)	45
5.4.7.5	自适应混合与结果合成	45
5.4.8	后期影像流水线与电影级合成	45

5.4.8.1	光学抖动还原与色调映射	45
5.4.8.2	曝光补偿与伽马校正 (Gamma Correction)	45
5.5	可视化与交互调试模块	46
5.5.1	渲染控制与参数动态绑定	46
5.6	本章小结	47
第六章	系统实现与测试	48
6.1	系统开发与运行环境	48
6.2	核心功能模块实现要点	48
6.2.1	基于 RAII 与延迟队列的资源管理实现	48
6.2.2	混合渲染管线的通道化压缩实现	48
6.3	系统测试	49
6.3.1	功能性测试用例	49
6.4	渲染效果对比与质量评估	50
6.4.1	SVGF 降噪效果分析	50
6.4.2	TAA 抗锯齿效果评估	51
6.5	性能定量评估与分析	51
6.5.1	各阶段 GPU 耗时统计	51
6.6	本章小结	52
第七章	总结与展望	53
7.1	全文工作总结	53
7.2	研究不足与未来展望	53

第一章 绪论

1.1 研究背景及意义

渲染是指通过软件算法将包含几何、视点、纹理和照明等信息的三维（或二维）模型转换为最终二维图像的技术。实时渲染是聚焦于制作和分析实时图像的计算机图形学的子领域。观察者在屏幕中看到的图像做出反应和操作，这些反馈会对下一张生成的图像产生影响。这种包含反应和渲染的循环会以一个很高的速度发生，以至于观察者没有反应过来自己在看一系列图像，而是沉浸在一个动态的过程中。通常会使用每秒显示的帧数（frames per second, FPS）或者赫兹（Hertz, Hz）来衡量图像显示的速率。一般达到 30 帧可以称为实时渲染，到 10 帧称其为可交互的渲染；而离线渲染的单帧渲染时间可达数分钟至数小时，追求极致的视觉质量。目前广泛应用在强交互的 3D 游戏和带 3D 的应用中，例如，游戏，VR/AR，虚拟仿真等，有广泛的应用前景和商业价值。

在实时渲染领域，主要有两种渲染技术：光栅化和光线追踪。光栅化技术历史悠久，是指把图像显示在屏幕上的过程。因为契合 GPU 的 SIMD（Single Instruction Multiple Data）并行计算模型，光栅化能够以极高的效率处理大量的几何数据，适合于大规模场景的渲染，从而。然而，光栅化在处理复杂的光影交互（如软阴影、全局照明和反射）时存在局限性，难以实现物理上准确的视觉效果。光追技术则通过 ray tracing（射线追踪）来计算反射、折射、可见性以及阴影，从而实现更精确的视觉效果。由于光线采样的随机性，难以契合 GPU 的 SIMD 并行计算模型，并且需要多次迭代，耗时较长，主要用于离线渲染领域。

随着 GPU 硬件能力和图形算法的飞速进步，实时光线追踪技术已逐渐从实验室走向工程实践。混合渲染是一种结合了多种渲染技术的方法，能结合旨在能结合光线追踪和光栅化各自的优势，也是目前主流的研究方向。

1.2 研究现状

图形渲染管线经历了长期的演进过程。20 世纪 90 年代末，开发者只能通过图形 API 提供的固定流程处理图像。随着统一着色器架构（Unified Shader Architecture）、GPU 驱动的几何剔除以及瓦片渲染（Tiled Rendering）等技术的出现，GPU 的自由度不断提升，图像编程变得愈发灵活。

2018 年是实时渲染的分水岭，Microsoft 发布了 DXR（DirectX Raytracing）框架，NVIDIA 推出了基于 Turing 架构的 RTX 平台，让实时光线追踪成为可能。。Vulkan 作为新一代跨平台图形 API，由 OpenGL 的开发团队 Khronos Group，打造的现代图像学 API，也提供了官方的光追扩展，本次引擎采用该 API 完成渲染构建。

由 ID SoftWare 开发的《德军总部 3D（Wolfenstein 3D）》《毁灭战士（DOOM）》取得了巨大成功，游戏引擎的概念也由约翰卡马克创建。DOOM 引擎是第一个用于商业授权的引擎，后来开发的 Quake 引擎是真正的 3D 引擎，人们普遍认为游戏《雷神之锤》带来了独立 3D 显卡的革命。如今的商业游戏引擎市场基本上是 Unreal, Unity 两巨头，在开源领域也有 Orge, Godot 等引擎，给许多引擎和渲染开发者提供了优秀的学习资料，促进了游戏引擎，渲染领域的发展。

除了渲染引擎整体的研究，也有对实时渲染相关问题的研究。实时渲染需求时效性，渲染质量论不如离线渲染，目前是在保证时效性下提高渲染质量。即使现在的硬件发展革新了实时渲染领域，在半透明等领域还，需要光线大量采样，目前 GPU 的算力仍无法满足需求。实时渲染的光线追踪使用 temporal，以低开销通过从历史图像获取数据，之后通过滤波去除高噪点信息，获取较高质量的结果。SVGF，在时空间采样和滤波，能满足实时渲染下对性能的要求，也能获得较好的图像效果。

1.3 本文研究内容

本文基于 Vulkan 1.3 规范设计并实现了一个三维实时混合渲染引擎，主要研究内容包括以下几个方面。

1. 研究 Vulkan 1.3 渲染管线，利用动态渲染、同步 2.0 等新特性优化引擎底层架构，简化资源管理流程。
2. 设计并实现基于渲染图（Render Graph）的资源调度模块，实现显存资源的自动生命周期管理与高效屏障同步。
3. 构建混合渲染管线，结合光栅化的高效几何处理能力与光线追踪的物理精确照明，实现高质量的软阴影与反射效果。
4. 研究并实现实时降噪与时空重构算法，针对低采样率的光线追踪信号进行处理，提升最终图像的视觉质量。

1.4 论文总体结构

以下是对本论文各个章节的概要：

- **第一章引言：**介绍实时混合渲染的研究背景、国内外现状，并简述本文的主要工作。

- **第二章理论基础：**深入分析 PBR 物理模型、Vulkan 渲染机制及硬件光追原理。
- **第三章需求分析：**从可行性、业务流程、功能与非功能指标等方面详细阐述引擎需求。
- **第四章概要设计：**论述系统分层架构、功能模块划分及显存逻辑表结构设计。
- **第五章详细设计：**以需求为基础，对渲染图算法、光追逻辑及时空重建模型进行细致设计。
- **第六章系统实现与测试：**展示核心代码实现，通过功能测试用例与性能数据验证系统有效性。
- **第七章总结与展望：**总结全文工作并指出未来改进方向。

1.5 本章小结

本章阐述了混合渲染引擎的研究意义，梳理了行业现状，并简要介绍了本文的主要研究内容。最后，本章给出了论文的整体章节安排，为后续内容的展开提供了总纲。

第二章 实时混合渲染相关理论

本章将阐述构建实时混合渲染引擎所需的理论基石。首先从宏观视角定义渲染器及其对物理世界的简化模型；随后深入探讨图形光栅化与光线追踪的基本原理；接着论述实时混合渲染管线的“分而治之”策略；最后探讨现代图形 API 的演进及其在渲染引擎架构中的核心作用，为后续章节中引擎的具体实现奠定严密的理论基础。

2.1 实时渲染相关基础理论

2.1.1 渲染模型

渲染（Rendering）是将离散的数字描述转化为最终可见像素阵列的过程。为了在有限的算力下模拟复杂的物理世界，本项目对渲染模型进行了三层维度的抽象简化：

- **相机模型：**将人眼视觉简化为针孔相机（Pinhole Camera），通过投影矩阵建立三维场景与二维成像平面的几何映射。
- **几何与材质模型：**利用三角形网格表征宏构拓扑，并采用基于微面元理论的材质参数（如粗糙度、金属度）描述物体表面的微观光学特性。
- **光源模型：**将发光体简化为点光源、方向光或具有解析分布的面积光源（Area Light），旨在平衡计算开销与光影的物理保真度。

2.1.2 渲染器职责与架构设计哲学

渲染器是图形系统的核心中枢，负责高效组织资源并驱动 GPU 指令流。现代高性能渲染器需遵循“显式声明、隐式同步”的设计原则。通过引入渲染图（Render Graph）技术，系统能够将高层算法逻辑与底层资源生命周期解耦，从而在保障执行效率的同时，降低由于显式 API（如 Vulkan）手动同步带来的研发风险。

2.2 图形光栅化原理

2.2.1 光栅化渲染管线流程

光栅化（Rasterization）是现代实时图形学的核心算法，其基本思想是将连续的几何图元（通常为三角形）投影并转化为离散的像素阵列。光栅化管线通常分为以下核心阶段：

- **几何阶段：**顶点着色器（Vertex Shader）执行模型、观察与投影变换，将三维坐标映射至齐次裁剪空间。随后进行裁剪与背面剔除，确保仅渲染视锥体内的可见图元。

- **光栅化阶段：**硬件对投影后的三角形进行扫描，确定哪些像素中心被覆盖，并利用重心坐标（Barycentric Coordinates）执行透视校正插值，为后续着色提供深度、法线及材质属性。
- **像素处理阶段：**片元着色器（Fragment Shader）计算最终色彩。通过深度测试解决遮挡关系，并由混合器完成最终的图像合成。

2.2.2 重心坐标插值与透视校正

在光栅化过程中，为了在三角形内部恢复顶点的属性（如 UV 坐标、法线），系统采用重心坐标插值。对于三角形内任意一点 P ，其属性 ϕ_P 可表示为三个顶点属性的加权平均：

$$\phi_P = \alpha\phi_A + \beta\phi_B + \gamma\phi_C \quad (2.1)$$

由于投影变换是非线性的，直接在屏幕空间进行线性插值会导致纹理扭曲。现代 GPU 强制执行透视校正插值（Perspective Correct Interpolation），通过对属性除以深度值 w 进行线性插值后再还原，确保了宏观透视关系的正确性。

2.2.3 硬件加速与 Early-Z 优化

为了进一步提升性能，现代 GPU 引入了 Early-Z 技术。在执行耗时的片元着色器之前，硬件会预先进行深度测试。如果当前像素已被遮挡，则直接跳过着色计算。这极大缓解了高复杂场景下的过度绘制（Overdraw）问题。在混合渲染中，这一特性常与 G-Buffer 预通行（Z-Prepass）结合使用，以最大化渲染效率。

2.3 光线追踪原理

2.3.1 光线投射与递归追踪

光线追踪（Ray Tracing）基于几何光学的可逆性原理。其基本流程是从相机（视点）向成像平面上的每个像素投射射线（Primary Ray），通过计算射线与场景中几何体的最近交点来确定可见性。Whitted 风格的递归光线追踪进一步模拟了光影的物理行为：在交点处，根据材质属性递归地发射反射射线、折射射线以及向光源发射阴影射线（Shadow Ray）。这种方法能够天然地处理复杂的镜面反射、透明折射和硬阴影，弥补了光栅化在全局视觉效果上的缺失。

2.3.2 光线追踪的计算挑战

与光栅化遍历几何体的逻辑相反，光线追踪是“遍历像素”的过程。其核心开销在于高频次的“射线-几何体求交”测试。在包含百万级三角形的场景中，线性求交的复杂度不可接受，因此必须引入高效的空间加速算法。虽然光线追踪在离线渲染（如影视

制作）中已应用多年，但其天然的非相干性与庞大的计算量，直到近年来硬件加速技术（RT Core）的出现才真正具备了实时执行的可能性。

2.4 实时混合渲染管线

2.4.1 分而治之的渲染策略

实时混合渲染（Hybrid Rendering）旨在结合光栅化的极高吞吐量与光线追踪的物理真实感。其核心逻辑在于根据视觉效果频率特性与计算成本，将渲染方程解构为不同的子项：

- **初级可见性：**利用光栅化快速生成 G-Buffer（包含深度、法线、反照率等），这避免了初级射线求交的庞大开销。
- **次级光影交互：**对于光栅化难以处理的非局部交互（如非线性软阴影、环境遮蔽及屏幕空间外的镜面反射），采用硬件加速的光线追踪进行离散采样。

2.4.2 管线集成与延迟渲染架构

混合管线通常建立在延迟渲染（Deferred Rendering）或分块/分簇渲染（Tile/Clustered Rendering）的基础之上。通过 G-Buffer 提供的空间几何上下文，光线追踪发射器（Ray Emitter）可以从像素位置出发，在世界空间内执行非相干的射线查询。最终，光栅化生成的直接照明与光线追踪生成的复杂光影项在合成阶段（Compositing）进行加权聚合，从而在保证实时性的前提下，实现接近路径追踪的效果。

2.5 图形 API 与渲染引擎

2.5.1 显式图形 API 的技术特征

现代图形 API（如 Vulkan、DirectX 12）标志着从“驱动代劳”向“开发者完全控制”的范式转变。其核心特征在于显式驱动机制：

- **显式资源生命周期：**开发者直接负责显存的分配、回收及镜像状态切换（Layout Transition），消除了传统驱动程序在后台进行推测性操作产生的隐式开销。
- **并行指令录制：**支持多线程并行录制命令缓冲（Command Buffer），彻底解决了传统 API 在 CPU 侧的单线程驱动瓶颈，极大提升了指令吞吐量。
- **精细化同步控制：**利用 Fence、Semaphore 和 Barrier 显式控制 GPU 内部任务的执行顺序与读写依赖（RAW/WAR），确保了光栅化与光追任务切换时的显存一致性。

2.5.2 渲染引擎的抽象层职责

为了平衡开发效率与硬件性能，现代渲染引擎不仅是 API 的封装器，更是复杂系统的调度中枢。其核心职责包括：

- **Bindless 资源管理:** 通过描述符索引（Descriptor Indexing）技术，将海量场景资源（纹理、材质缓冲区）一次性绑定至全局资源池。着色器通过索引实现 $O(1)$ 复杂度的随机访问，彻底消除了传统渲染中频繁切换状态产生的 CPU 开销。
- **自动化任务调度 (Render Graph):** 渲染引擎通过构建有向无环图（DAG）来描述渲染任务。Render Graph 能够自动推导资源的生命周期、自动插入同步屏障并执行显存别名优化（Aliasing），使开发者能专注于混合渲染算法的逻辑实现。
- **跨硬件特性适配:** 引擎通过特征检测机制，根据硬件能力动态启用光线追踪扩展（如 `VK_KHR_ray_tracing_pipeline`）或动态渲染（Dynamic Rendering），确保管线在不同代际 GPU 上的兼容性与稳定性。

第三章 需求分析

本章将从软件工程的角度对混合渲染引擎进行需求分析。通过对可行性、业务流程及功能/非功能需求的梳理，确立系统在图形表现、调度效率及用户交互方面的具体基准。

3.1 可行性分析

3.1.1 经济可行性

本项目完全基于开源生态构建，核心开发环境与依赖库均具有较高的经济效益：

- **开发成本：**Vulkan SDK、CMake、编译器（MSVC/GCC）以及图形驱动开发工具均可免费获取，无需支付商业许可证费用。
- **开源库集成：**集成了 GLFW、Dear ImGui、GLM 及 VMA 等开源领域流行的库，本身的开源证书限制较小，不要求支付版权费用。
- **硬件成本：**随着实时光线追踪显卡（RTX 系列）的普及，主流消费级硬件即可作为研发平台，研发设备投入处于合理区间。

3.1.2 技术可行性

- **标准支持：**系统基于 Vulkan 1.3 标准及 KHR Ray Tracing 扩展，利用 C++20 现代特性实现。当前硬件架构已提供成熟的硬加速求交单元（RT Core）。
- **知识获取：**Vulkan 的官方文档、开源引擎的源码以及学术论文为技术实现提供了丰富的参考资料。
- **开发工具：**目前的 Visual Studio, Visual Studio Code 等集成开发环境都能提供良好的开发环境，各种开源库也能支持引擎的资源加载，图像界面，输入输出等功能。利用 Vulkan 验证层（Validation Layers）及 RenderDoc 调试工具，方便调试。

3.1.3 操作可行性

- **交互直观性：**系统集成可视化 UI 调试面板，用户可实时通过 UI 界面，查看 Debug 信息，加载模型，移动光源，设置调试参数，以快速验证渲染的效果。
- **交互控制：**参考成熟的游戏引擎的相机，交互的实现，降低用户的操作门槛。

3.2 系统业务流程分析

本混合渲染器的业务流程如下所述：

1. **运行应用：**启动渲染器，初始化 Vulkan 上下文与图形驱动环境，加载窗口界面。
2. **场景加载与解析：**用户通过 UI 界面选择 3D 模型和天空盒，引擎会后台自动进行资源加载与解析。
3. **渲染节点构建：**系统根据用户选定的渲染路径（光栅化、混合路径或光追路径）自动构建 RenderGraph 拓扑图，分配资源和 Pass。
4. **交互调优：**用户通过菜单栏切换不同的渲染模块，实时调整光线方向，SVGF、TAA 开关，通过键盘鼠标调整相机位置和远近。
5. **数据反馈与可视化：**系统在主界面生成图像，同时可以选择中间视图，方便查看每个阶段的效果，以及耗时统计等 Debug 信息。

3.3 系统功能性需求分析

混合渲染器在设计时以图形开发者和环境艺术家为核心用户，基于其在场景搭建、效果调优及性能监控中的需求进行功能分析。以下 3.3.1 小节详细描述了用户的具体功能需求。

3.3.1 用户功能需求

用户作为系统的核心操作者，应具有如下功能：

3.3.1.1 场景与资产管理模块

该模块负责对渲染场景中的 3D 资产及其空间属性进行直观管理：

- **资源浏览与加载：**系统自动扫描指定目录下的模型与环境贴图。用户通过资源浏览器（Content Browser）可在多个 GLTF 模型间切换加载，并支持一键更换 HDR 环境天空盒。
- **场景层级管理：**用户通过层级面板（Hierarchy）实时查看当前场景中的所有实体对象，支持选中特定实例以进行后续操作，并提供安全移除选定实体的功能。
- **变换编辑：**支持对选中实体的平移（Position）、旋转（Rotation）和缩放（Scale）参数进行实时数值调节，并同步更新底层的硬件加速结构（AS）。

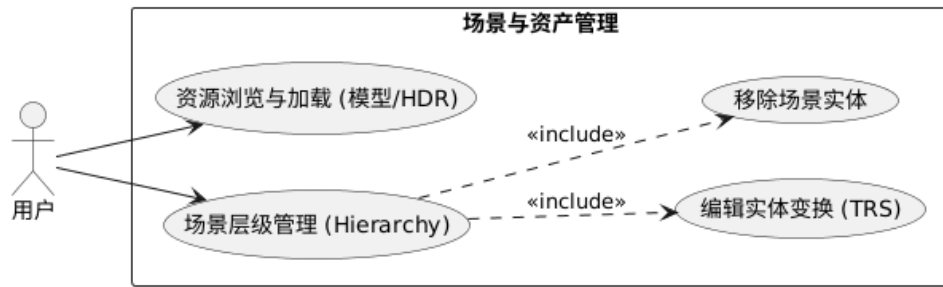


图 3.1 场景与资产管理用例图

3.3.1.2 Debug UI 界面

用户可以对渲染管线和环境光照进行深度定制，以达到最佳的画质表现：

- **渲染路径与视图切换：**用户可以在光栅化（Forward）、混合渲染（Hybrid）及路径追踪模式间实时切换。提供多种可视化模式，允许用户查看 Albedo、法线、材质参数、运动矢量以及 SVGF 方差图等中间附件。
- **渲染调试控制：**提供对光线追踪阴影、反射等独立控制的开关。针对信号重建，支持对 SVGF 时空降噪，TAA 抗锯齿等开关以查看效果。。
- **光源控制：**支持对主光源的方向、颜色进行调节，加载环境光照。
- **Debug 信息查看：**提供相机参数，帧数统计，性能分析等信息的实时显示，显示每个 Pass 的耗时，以及渲染管线拓扑图的的导出。

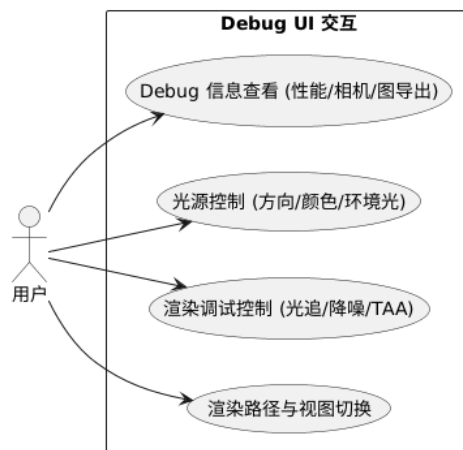


图 3.2 渲染控制与参数配置用例图

3.3.1.3 场景交互与观察模块

该模块为用户提供灵活的场景巡检手段与实时的系统状态反馈：

- **复合相机交互：**系统实现了一套基于弧球（Arcball）逻辑的编辑器相机。用户可通过“键盘 + 鼠标”的组合键模式进行操作：按住 Alt 键配合鼠标左键实现视角的环绕旋转，配合中键实现焦点的平移，配合右键或滚轮实现焦距缩放。该模式确保了用户能以任意角度观察场景细节。
- **交互状态反馈：**Debug UI 面板实时同步并显示相机的物理参数，包括相机的世界坐标（Position）、观察焦点（Focal Point）以及视场角（FOV），方便用户在调试特定渲染效果时精确记录机位。
- **系统性能监测：**集成实时的性能分析器，显示当前帧耗时（Frame Time）与帧率（FPS）波动。通过内置的时间戳查询技术，系统能够将渲染管线的内部执行细节可视化，允许用户监控每个渲染 Pass 的 GPU 执行开销及占比。

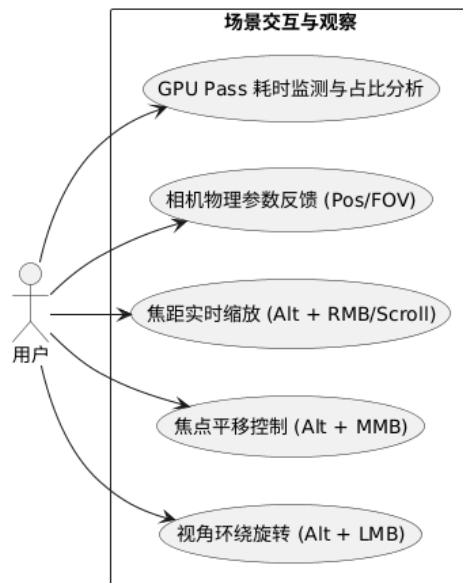


图 3.3 场景交互与观察用例图

3.4 系统非功能性需求分析

非功能性需求定义了系统在执行功能时所必须遵循的质量约束与技术指标。针对实时混合渲染系统的特性，本项目从性能、安全性、可用性、易用性以及可维护扩展性五个维度确立了非功能需求基准。

3.4.1 性能与响应时间

实时渲染系统对计算效率有着严苛的要求，需确保算法逻辑与数据流转的高度优化：

- **交互实时性指标：**在目标硬件平台上，系统应能维持稳定的亚秒级反馈。单帧渲染的端到端延迟应满足实时渲染的工业标准，确保在复杂场景漫游与参数调节时具有平滑的交互响应。
- **算法复杂度控制：**系统需针对计算密集型任务（如时空信号降噪）进行时间与空间复杂度的深度优化。利用并行计算等技术规避性能瓶颈，确保系统性能不随场景复杂度的增长而出现非线性骤降。

3.4.2 安全性与稳定性

在底层图形 API 驱动下，渲染系统必须构建坚实的资源保护与执行同步机制：

- **底层资源安全：**系统需实现一套基于引用计数与延迟销毁的显存管理机制。通过 RenderGraph 的资源生存区间分析，确保 GPU 正在引用的缓冲区或图像资源不会被非法释放，从架构层面预防驱动层面的访问冲突。
- **执行同步健壮性：**系统需要规范管理异步流水线中的资源依赖。通过自动化同步引擎推导并生成最优的同步屏障，避免由于读写竞态导致的画面闪烁或硬件超时重置。

3.4.3 可用性与可靠性

系统应具备在复杂运行环境下保持稳定输出的能力，并提供有效的容错逻辑：

- **状态切换可靠性：**在动态切换渲染路径（如从光栅化切至光线追踪）或调整配置参数时，系统需具备健壮的状态机管理能力，确保管线重建过程中的显存状态转换合法，不发生崩溃。
- **异常监控与诊断：**系统需要有良好的监测系统，能够实时捕获硬件特性缺失（如不支持 RT 扩展）或着色器编译异常，并向开发者提供详细的底层诊断信息。

3.4.4 易用性

系统界面设计与交互逻辑应符合图形行业规范，降低操作者的认知负担：

- **交互直觉性：**参数调节面板与场景管理接口的设计应遵循数字内容创作工具的通用逻辑。常用渲染配置应提供直观的可视化调节手段，实现“所见即所得”的操作体验。
- **辅助支持能力：**系统需配备完善的实时日志监测与诊断工具，并提供符合行业标准的摄像机控制系统，确保用户能够低成本、高效率地进行场景观察与算法验证。

3.4.5 可维护性与可扩展性

渲染系统的架构设计应支持长期的功能迭代与模块替换：

- **高度模块化解耦：**系统需采用逻辑层与驱动层分离的架构。通过高度抽象的调度中枢，使得新的渲染算法节点能够以插件化的方式接入，而无需对核心同步框架进行破坏性修改。
- **架构透明度：**系统需维持清晰的代码结构，确保其内部的数据流向与组件协作关系易于理解，从而降低二次开发与性能调优过程中的维护成本。

3.5 系统核心类设计

本节通过一系列类图展示系统的静态设计结构，涵盖从全局框架到具体渲染调度的协作关系。系统采用高度模块化的 C++ 设计，其核心类协作主要分布在框架管理、资源调度、场景组织及渲染执行四个主要维度。

3.5.1 全局管理与框架类设计

系统以 Application 为单例核心，通过组合模式聚合 Window 和 VulkanContext 确立运行环境。分层框架利用 `std::vector<std::shared_ptr<Layer>>` 构建逻辑栈，支持 EditorLayer 等功能层级的一致性分发。其中 VulkanContext 对 VulkanInstance 与 VulkanDevice 进行了深度封装，为全系统提供统一的硬件能力抽象。

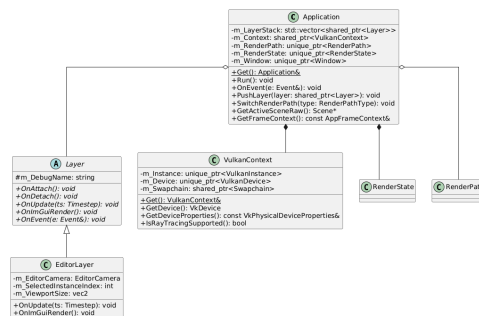


图 3.4 全局管理与分层框架类图

3.5.2 资源管理与场景组织类设计

系统采用组件化（Component-based）的场景组织结构。ResourceManager 负责 Vulkan 资源的 RAII 封装，通过基于线程池（Thread Pool）的多线程后台处理流水线，实现资产加载与逻辑更新的并行化，确保了大规模场景导入时的系统响应效率。Scene 类通过 Entity 容器组织场景对象，每个实体由 TransformComponent（TRS 变换）与 MeshComponent（模型引用）构成。系统通过对这些组件的实时追踪，自动维护并同步硬件加速结构（AS）。

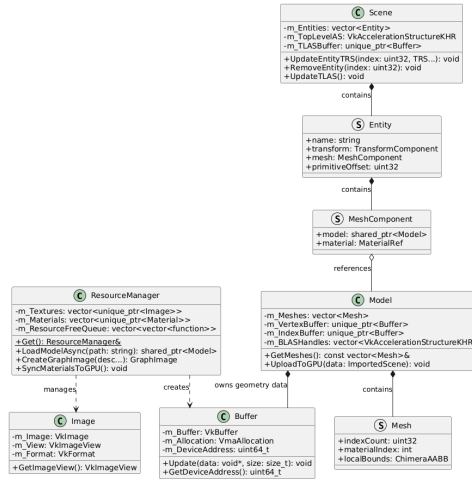


图 3.5 资源管理与场景组织类图

3.5.3 渲染调度引擎核心类设计

渲染调度由声明式的 RenderGraph 引擎驱动。系统通过 PassBuilder 捕获各渲染通道对显存资源的读写（Read/Write）依赖，从而推导出资源的物理生存区间。在执行阶段，引擎为不同类型的通道分发对应的执行上下文（ExecutionContext），屏蔽了底层同步屏障与镜像布局转换的复杂性，实现了高层次的架构解耦。

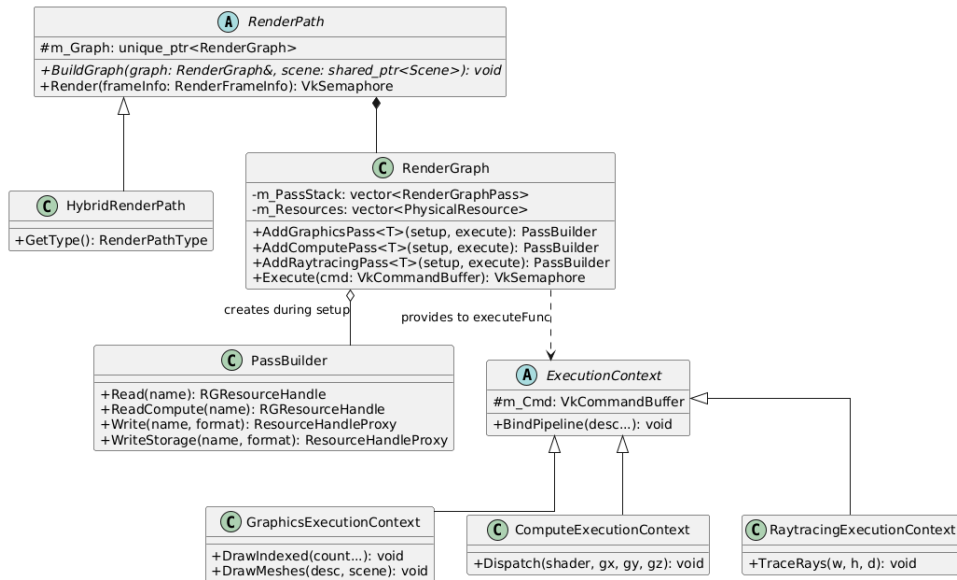


图 3.6 RenderGraph 调度引擎类图

3.6 本章小结

本章通过对可行性、业务流程及功能模块的深度拆解，明确了本渲染引擎混合渲染引擎的技术基准与交互标准。通过用例分析确立了用户在场景管理与管线调优中的核心需求，并通过详细的类图设计勾勒出了系统各子模块间的协作逻辑。本章的研究成果为下一章进入概要架构设计提供了明确的功能指导与工程依据。

第四章 概要设计

本章将对渲染引擎进行宏观架构设计。首先从渲染器在图形系统中的角色出发，确立系统的设计目标，随后详细阐述本项目采用的五层分层拓扑架构、核心模块划分以及 GPU 侧的资源组织方案。

4.1 渲染器设计概述

渲染器是游戏引擎中最核心的子系统之一，它负责图形绘制，接收渲染命令和资源数据（顶点、纹理、Shader 等），并调用 OpenGL/D3D/Vulkan 等图形 API，将游戏世界中的数据（模型、纹理、光照、摄像机等）转换为最终呈现在屏幕上的图像。渲染引擎的设计目标是构建一个具备现代工业级特性的混合渲染平台。在底层架构上，它追求与现代商业引擎相似的高内聚、低耦合特性，具备模型异步加载、资源自动管理及事件高效响应等基础能力，为上层复杂的混合渲染算法提供稳定的调度底座。

4.2 系统整体架构设计

4.2.1 架构总体思路

在系统搭建上，本项目参考了 Walnut 引擎的简洁设计理念，并在此基础上实现了基于 RenderGraph 的数据驱动架构。为了实现开发环境与引擎逻辑的物理隔离，系统采用“引擎核心 + 沙盒应用”的分离模式：

- **引擎核心 (Engine Core)**: 编译为静态库，封装了 Vulkan API 抽象、任务调度及渲染路径管理，屏蔽底层复杂性，是渲染引擎的核心内容。
- **沙盒应用 (Sandbox)**: 作为独立的可执行项目，通过调用核心库接口编写具体的渲染逻辑，开发者无需了解底层的同步与显存分配细节。

4.2.2 五层分层拓扑架构

依据现在游戏引擎的基本架构和对参考引擎 Walnut 的分析，本项目采用了如下分层拓扑架构。整个引擎从顶层应用到底层硬件划分为五个逻辑层，如图 4.1 所示。

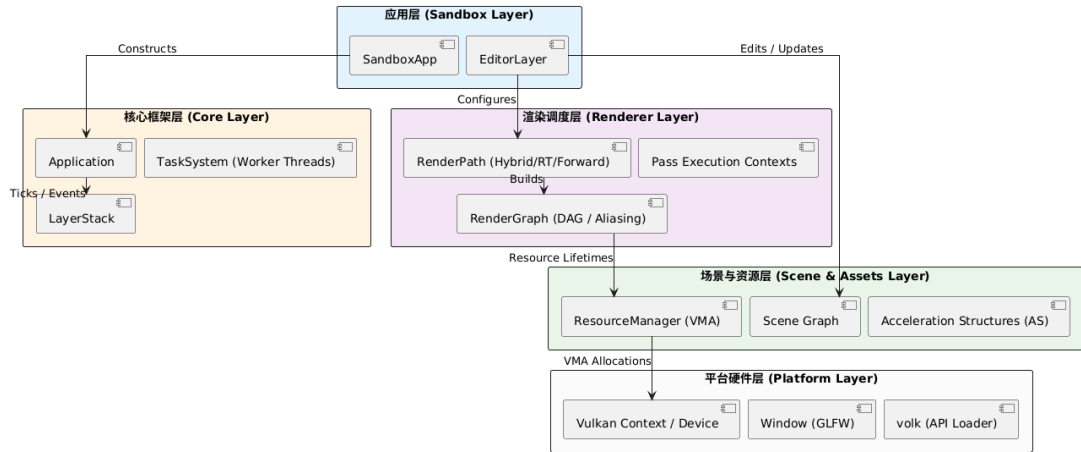


图 4.1 引擎五层分层拓扑架构图

1. **应用层 (Sandbox Layer)**: 具体的沙盒应用入口。开发者通过继承 Application 基类并挂载自定义逻辑层，实现不同渲染场景的快速构建。
2. **核心框架层 (Core Layer)**: 负责引擎的调度。包含支持插件式管理的 LayerStack、采用拦截机制的事件系统以及高层级的生命周期控制。
3. **场景与资源层 (Scene & Assets Layer)**: 负责资源和场景的管理。管理加速结构 (AS) 的实时更新、资源并行处理任务以及资源的中心化调度。
4. **渲染调度层 (Renderer Layer)**: 负责对渲染管线, Pass 等进行调度。以 RenderGraph 为中枢, 负责 DAG 构建、显存别名优化及指令录制。
5. **平台硬件层 (Platform Layer)**: 引擎的硬件抽象层。利用 volk 动态加载 Vulkan API, 负责交换链管理、窗口事件捕获及驱动级验证层配置。

4.3 核心功能模块划分

本节详细展示引擎六个核心模块的功能边界与逻辑职责。

4.3.1 核心框架模块

负责 Application 的生命周期与 LayerStack 管理。该模块集成事件系统实现“从顶到底”的事件拦截, 确保 UI 操作不会误触发场景交互。通过 Layer 抽象基类, 系统实现了逻辑的插拔式管理, 使得开发者可以灵活扩展 UI 界面或自定义调试工具。

4.3.2 资源管理模块

基于 RAII 机制封装 Vulkan 句柄, 核心组件 ResourceManager 采用单例模式统一管控所有图形资源。该模块通过延迟销毁队列 (Deletion Queue) 解决 GPU-CPU 异步释放冲突, 并集成 VMA 实现显存的池化管理。此外, 模块内置了多线程任务系统, 专

门负责 IO 密集型的 GLTF 异步加载与贴图并行解码。

4.3.3 场景管理与交互控制模块

采用实体-组件模型组织场景，实现基于 AABB 的视锥体剔除与静态八叉树空间划分。该模块通过 Scene 类统一管理 TLAS 实例，并内置 EditorCamera 实现弧球交互。交互系统利用输入轮询机制封装底层窗口状态，保障了复杂场景观察时的响应实时性。

4.3.4 渲染调度引擎模块

作为系统的调度中枢，该模块负责构建渲染任务的有向无环图（DAG）。其核心设计遵循“声明式渲染”原则，将渲染逻辑与底层驱动细节深度解耦：

- **虚拟资源映射机制：**开发者在注册阶段仅需使用逻辑 Handle 描述资源（如纹理、缓冲区）的读写意图，而不必感知具体的显存分配。系统在 Compile 阶段统一进行物理资源的绑定与别名优化。
- **拓扑分析设计：**通过对 Pass 节点进行拓扑排序，系统能够自动确立任务的执行序列，并基于资源生存区间分析（Liveness Analysis）实现显存的跨 Pass 复用。
- **屏障自动推导：**该模块通过追踪资源的 Layout 状态，自动在相邻 Pass 间插入最优的流水线屏障，彻底解决了手动管理同步导致的数据竞态问题。

4.3.5 混合渲染管线模块

实现“光栅化引导光线追踪”的混合管线流程。该模块整合了 G-Buffer 采集、硬件加速的光追查询（阴影、AO、反射及间接光）以及 SVGF 降噪算法。作为渲染路径的具体实现，它负责将复杂的算法 Pass 注入到 RenderGraph 调度引擎中执行，确立了“前级生产、后级消费”的数据依赖链条。

4.3.6 可视化与交互调试模块

基于 ImGui 实现全透明的实时监控界面。该模块提供渲染中间附件（如法线、方差图）的实时预览功能，并集成 GPU 性能戳统计分析工具，支持用户在运行时动态调节渲染参数以达到最佳画质平衡。

4.4 显存逻辑结构设计

系统针对 GPU 显存中的数据组织设计了以下逻辑结构，旨在优化高频次的数据访问：

- **全局材质属性表：**利用无绑定（Bindless）技术，将所有 PBR 参数（如颜色、粗糙度、贴图索引）线性化存储于全局 SSBO 中。着色器通过材质 ID 即可实现 $O(1)$ 复杂度的随机访问，彻底规避了传统描述符集频繁切换的开销。

- **资源元数据追踪表：**在 RenderGraph 内部维护一张资源状态表，记录每个纹理的格式、分辨率、生存区间及当前 Image Layout。这是实现自动化流水线同步与显存别名（Aliasing）复用的核心数据依据。
- **加速结构索引体系：**管理 TLAS 实例句柄与各 Mesh 对应的 BLAS 内存地址。该结构确立了几何数据在光线追踪求交计算中的空间映射关系，支持动态场景下的实时更新。

4.5 渲染数据流图 (DFD)

系统的渲染流程体现了从“CPU 资产描述”到“GPU 像素呈现”的高度自动化转换。其顶层数据流转逻辑体现了资产、逻辑与硬件指令的协同，如图 4.2 所示。

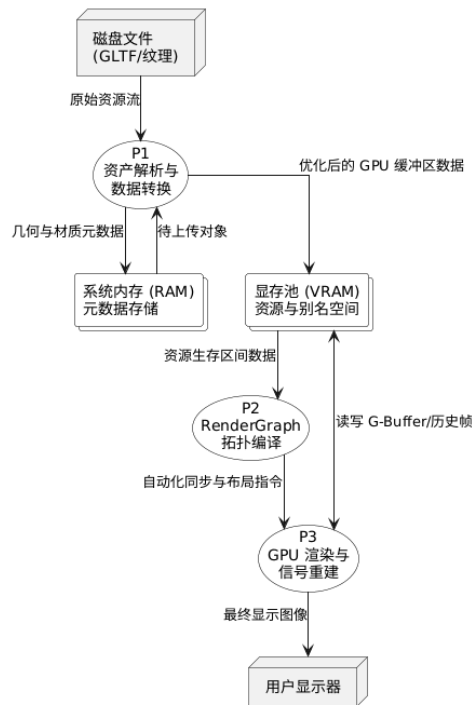


图 4.2 系统渲染数据流图 (DFD)

整个流转过程可分为三个核心阶段：

1. **资产预处理流 (CPU / TaskSystem)：**由多线程任务系统驱动，并行执行 GLTF 模型解析与贴图解码。随后在 GPU 侧分配物理显存并上传数据，同步完成底层 (BLAS) 与顶层 (TLAS) 加速结构的构建，为光追逻辑准备空间上下文。
2. **逻辑调度流 (RenderGraph Engine)：**调度中枢接收渲染路径的声明式配置，注册虚拟 Pass 节点及其资源依赖。通过拓扑排序确立执行序列，并基于生存区间分

析实现显存别名（Aliasing）优化与同步屏障（Barriers）的自动化插入。

3. **硬件指令流 (GPU Pipeline):** 引擎将逻辑节点转化为实际的 Vulkan 指令流。GPU 依次执行光栅化 G-Buffer 采集、硬件加速射线查询以及基于 SVGF/TAA 的画质重建算法，最终将混合渲染结果提交至交换链进行呈现。

4.6 本章小结

本章完成了引擎的宏观架构设计。通过五层分层拓扑架构确立了系统的解耦原则，并详细定义了六大核心功能模块的逻辑职责。显存逻辑与渲染数据流图的定义为后续详细设计中的具体算法实现提供了坚实的基础。

第五章 详细设计

在上一章节，通过对引擎架构的概要设计，确立了六个核心功能模块。本章将针对这些模块，详细阐述其底层的算法逻辑、核心类设计以及数据处理流程。通过对各子系统的深度拆解，将宏观架构具象化为可执行保持逻辑严密的工程方案。

5.1 核心框架模块

本项目的运行基石在于其高度解耦的框架层。本节将深入探讨硬件抽象层封装、应用生命周期与事件分发机制的底层实现。

5.1.1 Vulkan 环境初始化与硬件抽象

为了屏蔽 Vulkan 初始化过程中的复杂样板代码，引擎构建了一套轻量级的硬件抽象层。

- **动态加载机制：**利用 volk 库实现对 Vulkan API 入口的动态加载，绕过了静态链接驱动的局限性，提高了在不同厂商显卡间的运行兼容性。
- **设备与交换链抽象：**封装了 VulkanContext 单例，负责物理设备筛选、图形队列获取及交换链（Swapchain）的重建逻辑。
- **验证层与调试机制：**通过集成标准验证层（Validation Layers）并在 Debug 模式下激活调试回调（Debug Callback），系统实现了对 API 非法调用与资源同步竞态的实时监控。

5.1.2 Application 生命周期状态机

Application 类是引擎的单例中枢，其核心职责在于驱动主循环并分发窗口事件。

5.1.2.1 LayerStack 层级管理逻辑

为了实现模块化的 UI 与渲染控制，系统引入了 LayerStack。它将逻辑层划分为两类：

- **普通层 (Layer)：**按照添加顺序存储。底层的场景层先渲染，上层的辅助逻辑后渲染。
- **覆盖层 (Overlay)：**始终维护在容器末端。它们具有最高优先级，用于显示始终置顶的调试面板。

Layer 基类定义了一套标准的生命周期接口：

- **OnAttach/OnDetach**: 当层被推入或移出栈时触发, 用于资源的初始化与释放。
- **OnUpdate**: 每帧执行一次, 用于处理该层特有的逻辑更新与场景渲染指令提交。
- **OnImGuiRender**: 专门用于提交 ImGui 绘图指令, 将调试 UI 与核心场景逻辑物理分离。
- **OnEvent**: 接收分发的事件, 并根据逻辑决定是否消耗该事件。

5.1.2.2 基于“从顶到底”的事件拦截机制

为了实现平滑且高性能的交互响应, 系统设计了一套结合“事件驱动(Event-Driven)”与“输入轮询(Input Polling)”的混合交互架构。

事件分发逻辑与渲染顺序完全相反, 这种设计确保了 UI 交互的优先级。

1. **渲染流(从底向上, Bottom-to-Top)**: 在 OnUpdate 阶段, 引擎从 LayerStack 的最底层开始正向遍历。这确保了背景先绘制, 模型次之, 最后是覆盖在最上方的 UI 界面, 符合图形学的深度覆盖逻辑。
2. **事件流(从顶向下, Top-to-Bottom)**: 当鼠标点击或按键触发时, 系统调用 EventDispatcher 从 LayerStack 的栈顶(Overlay)开始倒序遍历。

这种“从顶向下”的传递机制实现了关键的**事件拦截(Event Interception)**。例如, 当用户点击 ImGui 面板上的按钮时, 顶层的 ImGuiLayer 会捕获该事件并将其标记为“已处理(Handled)”。此时, 分发器将立即终止循环, 下层的 SceneLayer 将不再收到该事件, 从而有效避免了在操作菜单时意外触发场景中物体的选择或移动。

5.1.2.3 输入轮询与连续交互逻辑

虽然事件系统能够精准捕获一次性动作(如切换渲染路径), 但对于需要每帧主动查询状态的“连续平滑响应”(如 WASD 相机漫游), 系统引入了输入轮询机制。

通过静态类 Input, 系统可以直接查询硬件的瞬时状态。该接口本质上是对底层 GLFW 输入状态的封装, 能够以与帧率同步的频率(与 OnUpdate 周期一致)获取输入, 从而实现无延迟的平滑移动体验。事件系统负责“决策与切换”, 而输入轮询负责“连续交互与操作”, 二者共同构成了系统的交互底座。

5.1.2.4 主循环逻辑详细设计

在 Application::Run() 方法中, 引擎通过死循环驱动。每一帧的核心执行步骤如下:

1. **Timestep 计算**: 计算两帧之间的时间差, 为相机移动与动画提供物理平滑的时间基准。

2. **输入轮询 (Input Polling):** 通过静态类 Input 实时查询键盘 (WASD) 与鼠标状态, 以支持高响应要求的交互。
 3. **层级更新与渲染:** 顺序执行各层的 OnUpdate 与 OnImGuiRender 接口。
 4. **缓冲区提交:** 调用 VulkanContext 的交换链呈现接口, 完成双缓冲切换。
- 系统每一帧的完整执行时序如图 5.1 所示。



图 5.1 Application 单帧执行生命周期活动图

5.2 资源管理模块

本模块是引擎性能与稳定性的核心。通过整合“中心化管理”与“异步任务流水线”，系统实现了大规模资产的流畅加载与显存的高效利用。

5.2.1 多线程任务系统与异步资源流水线

为了充分利用现代多核 CPU，系统构建了一套贯穿资源生命周期的并行架构。

5.2.1.1 TaskSystem 线程池调度逻辑

引擎在初始化阶段会创建一个常驻线程池 (Thread Pool)，其工作线程数量根据宿主机的硬件核心数动态分配。通过封装 `std::future` 机制，系统提供了一个轻量级的任务调度接口，允许主线程以非阻塞的方式分发耗时的计算任务。系统利用条件变量

（Condition Variable）实现高效的任務喚醒與睡眠機制，最大程度壓低 CPU 的空轉率。

5.2.1.2 后台异步资源处理流水线

在处理大规模复杂场景时，系统实现了一套四阶段异步加载流水线，将 IO 密集型与计算密集型任务从主循环中剥离：

1. **IO 与解析阶段：**在后台线程调用 Assimp 库读取 3D 原始文件并解析几何、材质元数据。相比于 cgltf，Assimp 提供了更强大的场景图处理能力和自动化的切线空间生成逻辑。
2. **并行解码与结构预处理：**利用线程池并行解码贴图，同时将原始顶点数据重组为 GPU 优化的 GpuTriangle 结构。该结构采用 vec4 紧凑布局存储位置、法线与切线，专为光线追踪中的高效率相交测试而设计。
3. **异步 GPU 上传：**在后台录制数据上传指令（Copy Command）并提交至独立的 Transfer 队列。
4. **延迟集成与材质映射：**主线程确认 GPU 操作完成后，执行“局部到全局”的材质索引映射。通过 UpdateMeshes 接口确保模型实例在渲染时能准确引用全局资源池中的材质属性。

整个异步加载任务的协作时序如图 5.2 所示。

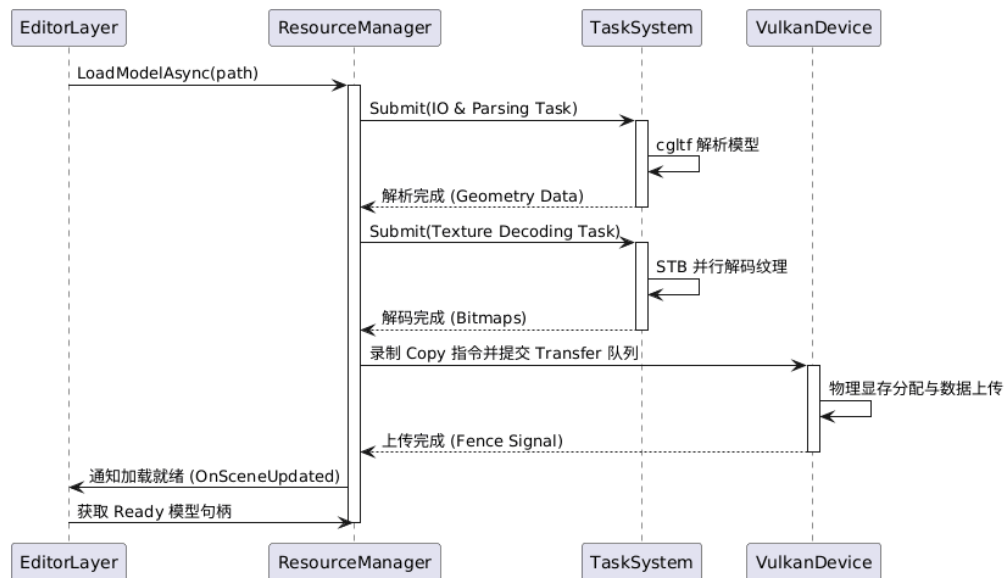


图 5.2 异步资源加载处理逻辑顺序图

5.2.2 ResourceManager 与延迟销毁逻辑

为了规避原始 Vulkan 句柄带来的管理负担，系统实现了一套高度抽象的“中心化资源调度系统”，通过单例模式的 ResourceManager 统一管控所有图形资源的生命周期。

5.2.2.1 资源对象抽象与 RAII 封装

引擎将琐碎的 Vulkan 底层对象封装为高级 C++ 类（如 Buffer、Image）。

- **RAII 管理:** 利用 C++ 的构造与析构函数。在对象析构时，系统自动调用 vkDestroy 并协同 VMA 释放显存，从架构层面杜绝了显存泄露。
- **GPU 物理布局对齐:** 为了实现与光线追踪算法的高效对接，系统严格定义了与 Shader 对齐的 C++ 结构体（如 GpuMaterial, GpuInstance）。这些结构体遵循 std430 对齐规范，确保了数据在 CPU 写入与 GPU 读取时的一致性。
- **材质数据驱动设计:** Material 类封装了基于 PBR 定义的材质参数。通过内部维护的“脏标记（Dirty Flag）”机制，系统仅在参数发生变化时才触发 GPU 端的同步，有效减少了冗余的显存拷贝。

5.2.2.2 基于帧序号的延迟销毁队列 (Deletion Queue)

Vulkan 严禁在 GPU 仍在执行渲染指令时销毁其引用的资源。由于 GPU 指令流与 CPU 调度是异步执行的，ResourceManager 引入了延迟销毁机制。当用户请求释放资源或实时移除场景实体时，系统不会立即执行物理销毁，而是将资源句柄及关联的 shared_ptr 推入延迟队列，并记录当前帧号。

系统会等待一个完整的渲染循环（通常为 3 帧，对应双重或三重缓冲），确保 GPU 已经彻底执行完相关指令后，才在主循环的末尾执行物理清理。该机制在处理“运行时动态删除”时具有关键意义，确保了即便当前帧的指令流中仍持有该资源的引用，系统也能稳定运行而不发生驱动报错或崩溃。

5.2.3 VMA 池化管理与 Bindless 架构设计

在底层资源分配与绑定上，项目结合了现代 Vulkan 的显存分配优化与资源访问技术。

5.2.3.1 基于 VMA 的显存子分配逻辑

直接为每个小型纹理或缓冲区调用 vkAllocateMemory 会迅速耗尽系统资源。系统集成了 Vulkan Memory Allocator (VMA)，其核心逻辑是在大块预分配显存 (Heap) 上进行子分配 (Sub-allocation) 与碎片整理。Chimera 将其封装在核心资源类中，极大地

简化了 Staging Buffer 的创建与显存同步流程。

5.2.3.2 基于 Bindless 思想的全局描述符绑定

为了支持百万级面片与海量纹理的实时混合渲染，系统采用了 ****Bindless**** 资源绑定架构：

1. **材质存储缓冲区 (SSBO)**: ResourceManager 将场景中所有材质的 PBR 参数打包成一个连续 GpuMaterial 数组，并通过接口一次性上传至全局 SSBO。
2. **全局描述符数组**: 通过 VK_DESCRIPTOR_BINDING_UPDATE_AFTER_BIND_BIT 标志，引擎维护一个巨大的描述符数组，将所有纹理视图一次性绑定。着色器不再需要针对每个物体切换绑定，而是通过实例数据中的索引直接寻址。

在着色器中，程序通过材质结构体中的索引直接在全局数组中随机访问资源，彻底消除了传统光栅化中频繁切换绑定状态导致的驱动开销，实现了“一个 Draw Call 渲染全场景”的性能潜力。

5.3 场景管理与交互控制模块

场景系统定义了虚拟世界的拓扑结构。为了兼顾光栅化管线的提交效率与实时光线追踪的遍历性能，系统实现了“CPU 空间划分 + GPU 硬件加速”的双层架构。

5.3.1 场景组织与空间加速算法设计

5.3.1.1 基于组合模式的场景实体结构

引擎采用轻量级的组件化设计实现几何与逻辑的解耦：

- **Entity (实体)**: 场景基本单元，持有全局唯一 ID。
- **TransformComponent**: 维护 4x4 TRS 矩阵，并保存前一帧状态。该组件在渲染流水线中至关重要，它为 TAA 抗锯齿算法提供了关键的像素级运动矢量 (Motion Vector)。
- **MeshComponent**: 持有 Model 引用，作为 BVH 构建的原始几何输入。

5.3.1.2 视锥体剔除与动态八叉树同步

为了降低 CPU 端的指令提交开销，系统在每帧渲染前执行两级可见性判定：

1. **基于 AABB 的视锥体剔除**: 系统采用 Gribb-Hartmann 算法，直接从当前的 View-Projection 矩阵中实时提取 6 个裁剪平面方程。通过将物体的局部 AABB 变换至世界空间并与平面进行快速求交，直接跳过视野外的物体录制。
2. **动态八叉树 (Octree) 空间索引同步**: 当场景实体数量达到量级增长时，系统构建静态八叉树。通过递归子分全场景包围盒，将查询复杂度从 $O(N)$ 降低至近似

$O(\log N)$ 。为了保障在实体动态移除时的索引一致性，系统实现了“全场景索引校准”策略：在检测到实体被移除后，立即重新计算剩余实体的基元偏移（Primitive Offsets），并彻底销毁旧的树结构后重新构建，确保了复杂场景下频繁交互的稳定性。

5.3.1.3 硬件加速的双层 BVH 维护

针对实时光线追踪，系统利用 Vulkan KHR 扩展维护双层加速结构：

- **底层加速结构 (BLAS)**：存储模型三角形的原始 BVH，由驱动程序在模型加载时一次性构建。
- **顶层加速结构 (TLAS)**：存储场景实例引用。当物体发生平移或旋转时，系统仅更新 TLAS 变换矩阵，无需重构复杂的顶点数据，从而确保了光追查询的实时响应。

5.3.2 EditorCamera 交互算法与坐标变换

摄像机系统是用户观察虚拟世界的“眼睛”。在底层图形学实现中，摄像机并不以物理实体形式存在，而是表现为一系列坐标空间变换的数学抽象。

5.3.2.1 摄像机原理：操作倒转与观察空间

理解摄像机的核心在于“逆变换”逻辑：在虚拟世界中，我们习惯于认为相机在空间中移动，但在渲染流水线中，实际发生的是**整个场景相对于相机的反向运动**。

例如，当用户指令相机向左移动时，渲染引擎实际上是将场景中所有的物体向右平移。这种将物体从世界空间（World Space）转换到观察空间（View Space）的变换矩阵被称为**观察矩阵（View Matrix）**。它代表了相机位置与姿态的逆矩阵，决定了观察者如何“看待”这个场景。

5.3.2.2 透视投影与裁剪空间

为了将观察空间的三维数据映射到二维屏幕上，系统引入了投影矩阵。项目主要采用透视投影（Perspective Projection），其核心参数包括：

- **视场角 (FOV)**：决定了视野的广度。较大的 FOV 会产生广角效果，增强画面的透视感。
- **纵横比 (Aspect Ratio)**：视口的宽度与高度之比，确保图像在不同尺寸的窗口中不会发生拉伸变形。
- **近/远裁剪面**：定义了相机可观察的深度范围，超出此范围的几何体将被剔除，以节省计算资源并缓解深度冲突。

5.3.2.3 MVP 变换流水线推导

一个顶点从模型原始数据到最终屏幕像素，需要经过完整的变换链，其数学表达为：

$$V_{clip} = P \cdot V \cdot M \cdot V_{local} \quad (5.1)$$

其中：

1. **模型矩阵 (M)**：通过平移、旋转、缩放（TRS）将顶点从局部坐标系转换到世界坐标系。
2. **观察矩阵 (V)**：将世界坐标系中的物体转换到以相机为原点的观察坐标系。
3. **投影矩阵 (P)**：应用透视除法，将坐标转换到归一化设备坐标（NDC）空间。

5.3.2.4 弧球 (Arcball) 相机交互实现

系统实现了一套基于弧球（Arcball）逻辑的编辑器相机。与传统的 FPS 相机不同，它始终围绕一个**焦点（Focal Point）**进行观察，极大地方便了对三维模型的全方位审视：

- **轨道旋转 (Orbit)**：通过计算鼠标在屏幕上移动产生的偏航角（Yaw）和俯仰角（Pitch），结合四元数（Quaternion）旋转，实现围绕目标的平滑旋转，且有效规避了万向锁问题。
- **自适应缩放与平移**：系统根据相机到焦点的距离动态调整移动步长。这种自适应逻辑确保了在观察宏观场景（如 Sponza 整体）时具有较高的移动效率，而在接近模型微观细节时自动降低步长，从而在不同尺度下都能保持一致且细腻的操作手感。

5.3.2.5 基于 Reversed-Z 的深度精度优化

为了适配 Vulkan 的坐标系规范并缓解大规模场景下的深度冲突（Z-Fighting），系统在投影矩阵中修正了 Y 轴的方向，并应用了 Reversed-Z 技术。通过将近裁剪面映射为 1.0，远裁剪面映射为 0.0，并配合浮点深度缓冲，利用了浮点数在零点附近的更高精度，使整个视锥体内的深度精度分布更加均匀，极大地提升了画面稳定性。

5.4 渲染调度引擎 (RenderGraph) 详细设计

RenderGraph 是渲染引擎的调度中枢，它通过构建有向无环图（DAG）实现了渲染逻辑与底层同步的深度解耦。

5.4.1 渲染图核心原理与架构设计动机

5.4.1.1 自动化流水线工厂类比

为了更形象地理解 RenderGraph 的内部工作原理，可以将其类比为高度自动化的流水线工厂：

- **RenderGraph (中央调度系统)**：负责监控全局物料流向并规划每一道工序的生产时序。它并不参与底层的绘制计算，但决定了指令录制的逻辑顺序。
- **Pass (工作站)**：是生产的最小单元。每一个工作站通过“派工单”明确告知系统所需领取的半成品（Input）以及最终产品的存放位置（Output）。
- **Resource (物料)**：既是 Pass 提交的虚拟“提货单”，也是最终映射在 GPU 上的真实物理显存。
- **RenderPath (图纸)**：决定了整条流水线的拓扑构型，将一系列复杂的渲染 Pass 逻辑按预定序列提交给工厂。

这种设计哲学的核心在于：**显式声明依赖，隐式自动同步**。开发者只需关注“要做什么”，而系统负责处理“该怎么做”。

5.4.1.2 架构设计动机：规避“同步地狱”

在显式 API（如 Vulkan）开发中，手动管理数以百计的同步屏障（Barrier）与图像布局转换（Layout Transition）极易导致画面闪烁或驱动崩溃。引入 RenderGraph 的核心驱动力在于：

- **自动化同步推导**：系统通过分析 Pass 间的读写依赖（RAW/WAR），自动插入最优的同步指令，确保了 GPU 显存访问的安全性。
- **显存复用优化 (Aliasing)**：基于资源的生存区间分析，系统可以自动让生命周期不重叠的资源复用同一块物理地址，有效压低了显存峰值占用。

5.4.2 核心组件设计与架构模式

系统采用“外观模式（Facade）”与“策略模式（Strategy）”相结合的方案，由以下四个核心协作组件构成：

- **RenderGraph (调度中心)**：作为系统的中枢，它持有所有的渲染通道栈（`m_PassStack`）与物理资源池（`m_Resources`）。它驱动了从依赖收集到指令提交的完整生命周期。
- **PassBuilder (外观接口)**：这是开发者在注册阶段接触的唯一接口。通过外观模式隐藏底层的物理资源分配细节，用语义化的 `Read` 与 `Write` 方法，从架构层面保证了任务依赖关系的清晰。

- **RenderPass (任务单元)**: 负责存储具体的任务元数据, 包括 Pass 类型、着色器名称以及核心执行闭包。这种设计实现了管线拓扑构建与具体指令执行的物理解耦。
- **RenderGraphRegistry**: 在执行阶段, 该类作为物理资源的查询接口, 将逻辑 Handle 安全地转换为 `VkImageView` 等物理句柄。

5.4.2.1 指令录制优化: 基于静态多态的智能分发

在复杂的混合渲染管线中, 为了消除频繁实例化不同执行上下文产生的样板代码, 渲染引擎实现了一套基于静态多态 (Static Polymorphism) 的智能路由机制。利用 C++17 的 `if constexpr` 技术, 系统能够在编译期自动探测 Pass 类定义的 `Execute` 函数签名:

$$Dispatch(Pass) \rightarrow \begin{cases} GraphicsContext & \text{if } Pass \text{ signature matches } GraphicsContext\& \\ ComputeContext & \text{if } Pass \text{ signature matches } ComputeContext\& \\ RaytracingContext & \text{if } Pass \text{ signature matches } RaytracingContext\& \end{cases} \quad (5.2)$$

这种“编写即配置”的设计确保了录制逻辑的高性能, 开发者仅需声明所需的上下文类型, 系统便会自动处理底层初始化工作 (如 `vkBeginRendering`), 且能在编译阶段拦截非法的 API 调用。

5.4.3 运行生命周期与自动化调度逻辑

渲染图每帧会经历从虚拟描述到物理执行的三个关键阶段, 其完整生命周期流程如图 5.3 所示。

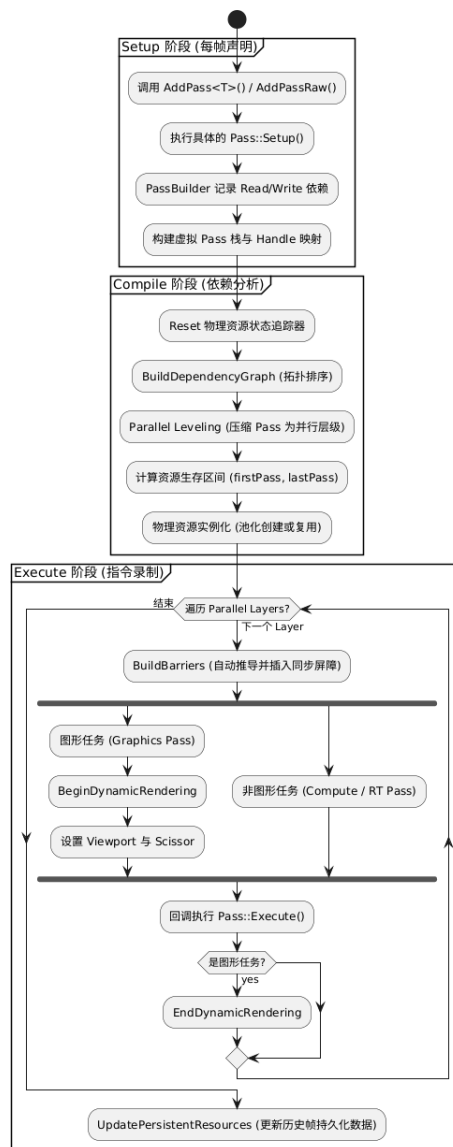


图 5.3 RenderGraph 运行生命周期流程图

5.4.3.1 Setup：虚拟依赖构建

在此阶段，主程序顺序调用各 Pass 的注册逻辑。开发者通过 PassBuilder 建立任务间的逻辑依赖关系。此阶段不分配真实显存，不录制 GPU 指令，仅仅是在内存中构建出一张“虚拟提货单”。

5.4.3.2 Compile：拓扑分析与显存优化

在执行前，RenderGraph 对拓扑结构进行高效编译：

1. **拓扑排序**：利用 DFS 算法计算出合法的执行序列，确保生产者始终在消费者之前运行，并自动剔除未被引用的冗余 Pass。

2. **生存区间分析 (Aliasing)**: 系统计算每个资源的 `firstPass` 与 `lastPass`。对于生命周期不重叠的资源, 系统通过显存别名技术使其复用同一块物理地址, 极大地压低了 MRT 带来的显存带宽压力。

5.4.3.3 Execute: 自动同步引擎

在执行阶段, 系统激活“自动同步引擎”。该阶段的自动化同步与指令录制时序如图 5.4 所示。

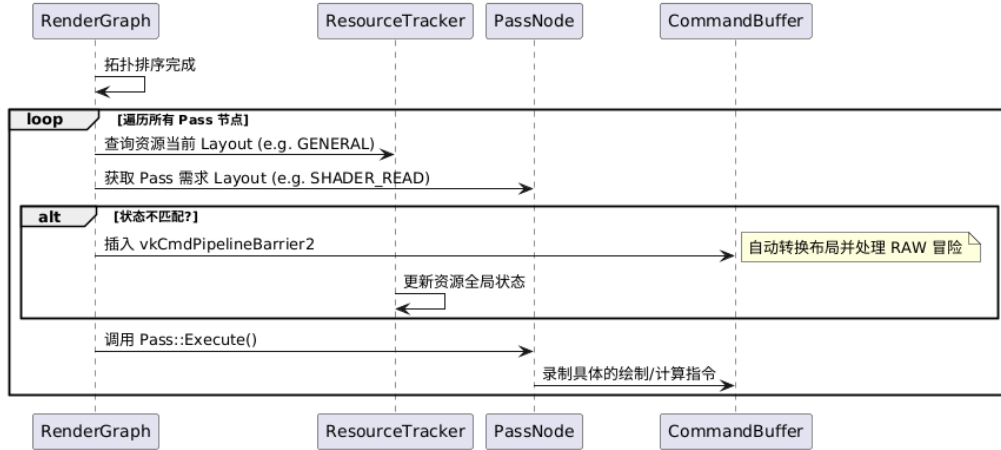


图 5.4 RenderGraph 自动同步与指令录制顺序图

系统实时对比资源在相邻 Pass 间的访问需求差异 (如从 GENERAL 布局转换至 SHADER_READ_ONLY)。若状态不匹配, 系统会自动生成最小集合的 `vkCmdPipelineBarrier2` 指令, 彻底解决了手动管理写后读 (RAW) 冒险导致的画面闪烁问题。此外, 系统通过插入 GPU 时间戳查询, 实现了对每个渲染阶段纳秒级的性能监控。

5.4.4 基于微面元理论的 PBR 物理光照详细设计

项目实现了一套基于物理 (Physically Based Rendering) 的评估体系, 其数学模型与先进的降噪器 SVGF 进行了深度对齐。

5.4.4.1 双 Lobe 材质评估模型

系统采用了 Disney 原则下的 PBR 架构, 将材质响应划分为两个独立的 Lobe。总体的渲染方程表达为:

$$f(i, o) = (1 - \text{metallic}) \cdot f_{\text{diffuse}}(i, o) + f_{\text{specular}}(i, o) \quad (5.3)$$

其中, 漫反射项 f_{diffuse} 采用 Lambertian 模型, 而镜面反射项 f_{specular} 采用 Cook-Torrance 微面元模型。

5.4.4.2 法线分布函数与几何遮蔽

- **法线分布函数 (D):** 采用 Trowbridge-Reitz GGX。该函数引入了粗糙度 α 的平方映射 ($\alpha = roughness^2$)，能够产生具有物理正确性的“长尾”高光边缘。其数学形式为：

$$D(h) = \frac{\alpha^2}{\pi((n \cdot h)^2(\alpha^2 - 1) + 1)^2} \quad (5.4)$$

- **几何遮蔽项 (G):** 采用 Smith 联合分布。针对 SVGF 降噪器的特性，系统应用了精确的影子项计算，确保在高角度下光影能量的自然衰减。

5.4.4.3 Kulla-Conty 多重散射能量补偿

为了解决微面元模型在高粗糙度表面因“单次散射”假设导致的变暗现象，系统引入了能量补偿项。通过预估被几何遮挡 (Shadowing-Masking) 而原本会丢失的能量，系统将其作为二次反弹贡献加回着色方程：

$$f_{total} = f_{single} + \frac{F_0 \cdot (1 - E_{avg}) \cdot (1 - E_i) \cdot (1 - E_o)}{\pi \cdot (1 - E_{avg})} \quad (5.5)$$

该补偿机制确保了如磨砂金属、毛玻璃等高粗糙度材质在物理上满足能量守恒，极大地提升了复杂光照环境下的真实感。

5.4.5 混合渲染管线与信号重建算法详细设计

混合渲染管线的核心在于利用光栅化的高效性为光线追踪提供精确的上下文。整个执行过程体现了从“确定性几何提取”到“随机性光影计算”的递进，其单帧内的六阶段执行时序如图 5.5 所示。

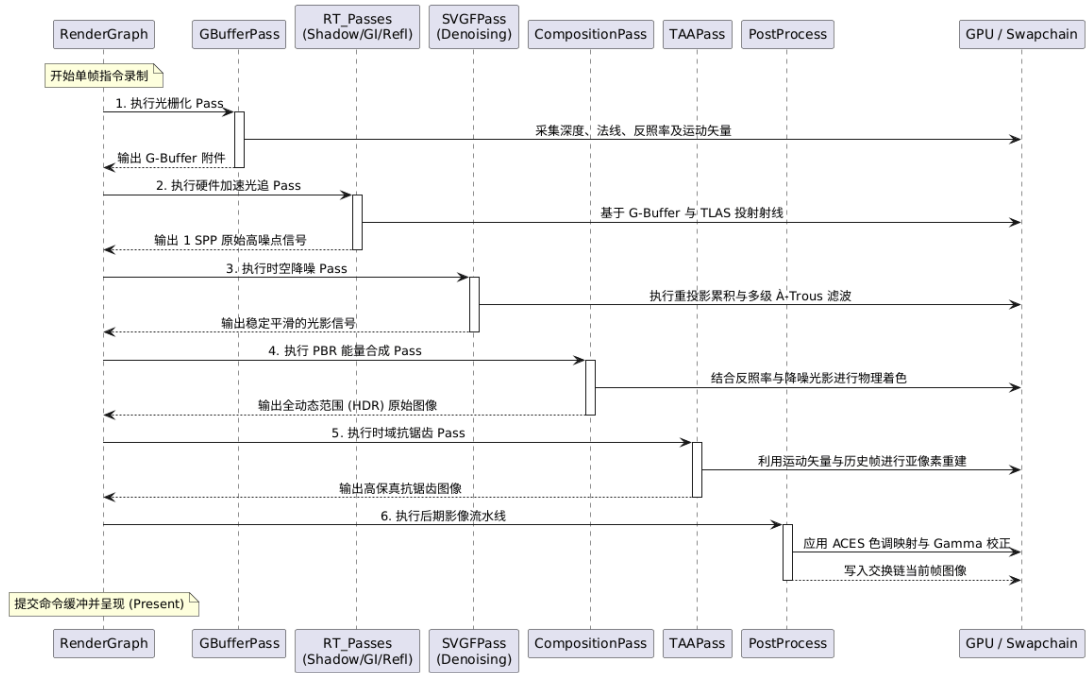


图 5.5 混合渲染管线单帧执行时序图

根据 RenderGraph 的调度逻辑，管线每帧严格执行以下六个阶段：

1. **GBufferPass**: 利用光栅化管线快速采集场景的深度、法线、反照率及运动矢量。这些几何附件为后续阶段提供了亚像素级的空间索引。
2. **RT_Passes**: 基于 G-Buffer 提供的世界坐标起始点，向光源及半球空间发射 1 SPP 射线。通过硬件加速结构（TLAS）捕获阴影、环境遮蔽及漫反射间接光等高噪声原始信号。
3. **SVGFPass**: 执行降噪核心逻辑。通过时域重投影累积历史信息，并利用多级 $\tilde{A}$ -Trous 跨步滤波器在空间域滤除蒙特卡洛方差噪声。
4. **CompositionPass**: 将降噪后的光影信号与反照率进行物理耦合，执行基于 Cook-Torrance 模型的光照合成，生成全动态范围（HDR）的原始着色图像。
5. **TAAPass**: 针对几何边缘的采样走样，利用亚像素抖动与时域反馈机制进行抗锯齿处理，进一步提升画面的几何稳定性。
6. **PostProcessPass**: 应用后期影像流水线。执行光学抖动还原、ACES 色调映射及伽马校正，最终将电影级视觉产出写入交换链。

5.4.5.1 SVGF 时域累积与矩估计逻辑

为了将 1 SPP 的高噪声原始信号转化为平滑的视觉产出，系统实现了 SVGF (Spatiotemporal Variance-Guided Filtering) 管线。其第一阶段为时域累积，通过对历史数据的重投影校验，在不增加射线采样数的前提下大幅提升了信号的统计有效性。

5.4.5.1.1 1. 鲁棒重投影校验 (Reprojection Validation) 系统利用 G-Buffer 中的运动矢量 $\mathbf{v}$ 将当前像素 $\mathbf{p}$ 映射至前一帧位置 $\mathbf{p}'$ 。为了识别遮挡关系破裂 (Disocclusion)，系统应用了严格的校验逻辑：

- **深度一致性:** $|Z_{curr} - Z_{prev}| < 0.7$ 。
- **法线一致性:** $\vec{N}_{curr} \cdot \vec{N}_{prev} > 0.9$ 。
- **物体 ID 校验:** 确保当前像素与重投影像素属于同一 Mesh 实例。

5.4.5.1.2 2. 亮度矩的时域累积 若校验通过，系统对颜色信号 $\mathbf{C}$ 与亮度矩 M 进行混合累积：

$$\bar{\mathbf{C}}_t = (1 - \alpha)\bar{\mathbf{C}}_{t-1} + \alpha\mathbf{C}_t, \quad \bar{M}_t = (1 - \alpha)\bar{M}_{t-1} + \alpha M_t \quad (5.6)$$

其中 $\alpha = 1/\text{HistoryLength}$ 。通过累积第一矩（亮度 L ）与第二矩（亮度平方 L^2 ），系统可以实时估计局部信号的方差： $\sigma^2 = \bar{L}^2 - \bar{L}^2$ 。该方差将作为后续空间滤波器的引导权重。

5.4.5.2 基于 Å-Trous 的多级边缘启发式滤波

在获得带方差引导的信号后，系统执行 5 遍 Å-Trous 空间滤波。

5.4.5.2.1 1. 统一边缘停止函数 (Edge-Stopping Functions) 对于每个采样样本 q ，其贡献权重 $w(p, q)$ 由卷积核权重 h 与三个启发式分量共同决定：

$$w(p, q) = h \cdot w_z \cdot w_n \cdot w_l \quad (5.7)$$

其中：

- **深度权重 w_z :** 利用深度梯度 ∇z 抑制跨深度的颜色渗漏。
- **法线权重 w_n :** 基于点积的幂函数，保持几何边缘的锐利度。
- **亮度权重 w_l :** 基于估计方差 σ 动态调整，定义为 $\exp\left(-\frac{|L_p - L_q|}{\phi\sigma + \epsilon}\right)$ 。在噪声剧烈区域， σ 较大，滤波器会自动增大模糊半径。

5.4.5.2.2 2. 方差的空间传播 在 À-Trous 迭代中, 不仅对颜色进行滤波, 对方差也进行同步更新。根据误差传播定律, 滤波后的方差计算公式为 $\sigma_{filtered}^2 = \sum w_i^2 \sigma_i^2 / (\sum w_i)^2$ 。这种递归式的方差估计确保了降噪过程在数学上的收敛性。

5.4.5.3 光栅化中的离散采样与锯齿现象

在光栅化阶段, 系统对每个三角形进行覆盖测试。由于像素中心点的分布是离散的, 当三角形的边缘斜向穿过像素阵列时, 生成的着色像素点会呈现断断续续的阶梯状, 这种现象被称为锯齿 (aggies)。在信号处理中, 这种由于采样频率不足导致的失真统称为**走样 (Aliasing)**。

走样的根本原因在于: 信号变化的频率过高 (高频信号), 而采样的频率过慢, 导致采样后的信号发生频谱混叠。为了减少走样, 通常遵循“先模糊, 再采样”的预滤波 (Pre-Filtering) 原则。典型的 G-Buffer 各层分量如图 5.6 所示。

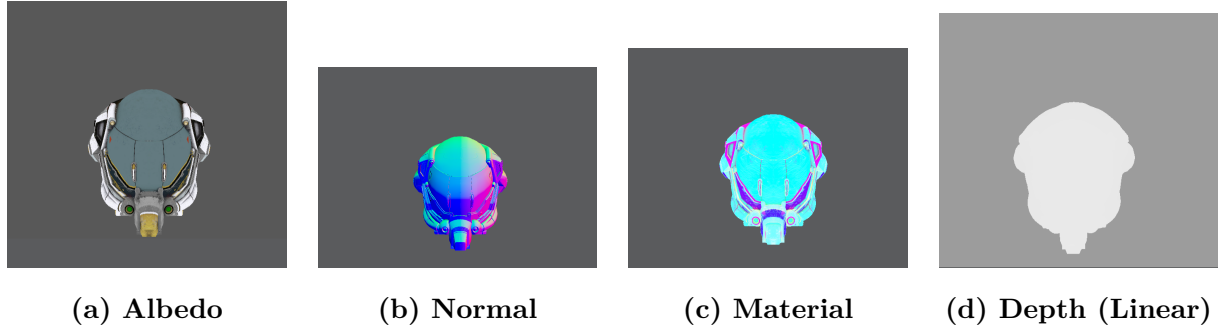


图 5.6 G-Buffer 多附件几何信号采集分量图

通过提取场景的深度 (Depth)、法线 (Normal) 及材质参数, 系统为后续的光线追踪管线提供了精确的求交起始点与表面属性依据。

5.4.5.4 蒙特卡罗积分与重要性采样逻辑 (CDF)

在混合渲染管线的光线追踪阶段, 求解渲染方程本质上是求解半球空间上的复杂积分。为了在极低采样率 (1 SPP) 下获得可接受的初始信号, 系统引入了基于累积分布函数 (CDF) 的重要性采样技术。

5.4.5.4.1 1. 蒙特卡罗估计与方差痛点 渲染方程的数值解通常通过蒙特卡罗估计器 (Monte Carlo Estimator) 获得:

$$F_N = \frac{1}{N} \sum_{i=1}^N \frac{f(x_i)}{p(x_i)} \quad (5.8)$$

其中 $p(x_i)$ 是采样点的概率密度函数（PDF）。若采用均匀采样，当场景中存在极小且极亮的光源时，采样值 $f(x_i)$ 的波动会导致估计器方差巨大，表现为画面上严重的“斑点”噪声（Fireflies）。

5.4.5.4.2 2. 重要性采样与 CDF 的推导 为了降低方差，理想的采样分布应尽可能接近函数 $f(x)$ 的形状。系统通过构建累积分布函数（Cumulative Distribution Function, CDF）将均匀分布的随机数 $r \in [0, 1)$ 映射到高权重的采样区域。

对于场景中的 M 个离散发光体（发射面或解析光源），其选择概率与光通量成正比。CDF 的构建过程如下：

1. **构建 PDF**：计算每个光源的权重 w_i （如三角形面积 $\times$ 发射强度），归一化得到 $p_i = w_i / \sum w$ 。
2. **构建 CDF**：进行前缀和累加， $P_k = \sum_{i=1}^k p_i$ ，其中 $P_M = 1.0$ 。
3. **逆变换采样**：在 GPU 端生成随机数 r ，通过二分查找（Binary Search）定位满足 $P_{k-1} \leq r < P_k$ 的索引 k 。

5.4.5.4.3 3. 硬件加速下的 NEE 策略 系统结合直接光源采样（Next Event Estimation, NEE）策略，在光路追踪的每个交点处，利用上述 CDF 显式采样一个光源并测试可见性。这种方法确保了即使在 1 SPP 条件下，降噪器收到的初始信号也具有较高的信噪比，极大地减轻了后续 SVGF 算法的压力。

5.4.5.5 实时光线追踪信号探测与采样算法设计

系统中的阴影、环境光遮蔽及间接光算法均以“RT”（Ray Tracing）为前缀，旨在强调其完全基于 **Vulkan KHR** 硬件加速接口实现。相比于传统光栅化时代的经验模型（如 Shadow Maps、SSAO 等），基于 RT 的探测算法具有以下显著特征：

- **硬件加速的 BVH 遍历**：算法利用 GPU 内置的 RT Core 进行加速，通过对 TLAS（顶层加速结构）的硬件级求交，取代了基于深度贴图的纹理采样，获得了亚像素级的空间位置精度。
- **视口外感知 (Off-screen Awareness)**：传统算法（如 SSR）仅能利用屏幕空间内的可见信息，导致屏幕边缘效果断裂。RT 算法能够感知相机视锥体之外的几何体，为反射与全局光照提供了完整的物理环境支撑。
- **从经验模型到随机统计的演进**：RT 算法采用蒙特卡洛（Monte Carlo）随机采样逻辑，将光照计算从单一的“解析经验值”转变为“统计期望值”。虽然这在 1 SPP

采样率下会产生蒙特卡洛噪声，但其天生支持物理正确的半影虚化与颜色反弹。

5.4.5.5.1 1. 实时光追软阴影探测 (Ray Traced Shadows) 传统的阴影贴图(Shadow Maps) 受限于分辨率且难以模拟物理半影。系统通过向面光源进行概率采样实现软阴影效果。

- 1. 锥形等概率采样模型:** 系统将方向光(如太阳光)抽象为具有一定张角 θ 的远景面光源。在探测阴影时,利用蓝噪声(Blue Noise)抖动生成样本 ξ ,通过 SquareToUniformCone 映射函数将射线方向 L_{dir} 约束在以光源主方向为中心的锥形范围内。这使得阴影边缘呈现出“接触硬化(Contact Hardening)”特征:即靠近物体的阴影锐利,远离物体的阴影随距离增加而自然虚化。
- 2. Any-Hit 硬件遍历优化:** 阴影射线仅需确认路径上是否存在“任意”遮挡,而无需计算最近交点。系统在 traceRayEXT 调用中激活 gl_RayFlagsTerminateOnFirstHitEXT 标志,使硬件 RT Core 在检测到首个碰撞基元后立即返回,极大地压低了在植被等高密度几何体场景下的射线遍历开销。

5.4.5.5.2 2. 光线追踪环境光遮蔽 (Ray Traced AO) RTAO 解决了传统 SSAO 无法感知屏幕外几何体遮挡的问题。其核心在于对局部半球空间的可见性积分:

- 1. 余弦权重采样 (Cosine-weighted Sampling):** 根据漫反射表面的物理特性,系统在表面法线 n 的半球范围内发射射线。射线的分布密度与 $\cos \theta$ 成正比,其数学期望遵循:

$$AO(p) = \frac{1}{\pi} \int_{\Omega} V(p, \omega) (n \cdot \omega) d\omega \quad (5.9)$$

其中 $V(p, \omega)$ 为二值可见性函数。

- 2. 短程探测设计:** 为了保证 AO 的局部特征(局部对比度),系统将射线的最大探测距离 T_{max} 限制在固定半径(如 2.0 单位)内。这使得墙角、缝隙等微小区域能产生深邃且自然的阴影感,显著增强了场景的体积感。

5.4.5.5.3 3. 1-Bounce 漫反射间接光 (Ray Traced GI) 间接光探测实现了光线在物体间的真实反弹,其设计包含两个关键步骤:

- 1. 1-Bounce 能量传输路径:** 间接光射线从第一交点(Primary Hit)出发,在 Closest Hit 阶段再次进行直接光计算。这一过程模拟了光线从“光源 $\rightarrow$ 二次反射点 $\rightarrow$ 主

表面 → 相机”的能量路径。相比于 0-Bounce 的环境光，它能产生显著的颜色流溢（Color Bleeding）效果。

2. **反照率解耦 (Albedo Demodulation) 机制：**这是系统针对混合渲染设计的核心优化方案。为了后续 SVGF 降噪器的稳定性，系统在输出 GI 信号前剥离了材质的基色：

$$I_{irradiance} = \frac{L_{radiance}}{\rho_{albedo} + \epsilon} \quad (5.10)$$

物理意义：间接光辐射度（Radiance）包含了高频的物体纹理，直接降噪会导致纹理模糊。通过解耦出的辐照度（Irradiance）仅代表空间光照场的低频分布，更适合进行时空滤波。最终在合成阶段再通过乘回 Albedo 进行重调制（Re-modulation），从而实现了在消除光追噪声的同时，保留物体表面纹理的绝对锐利。

5.4.6 SVGF 时空方差引导降噪算法实现

针对 1 SPP 采样率下极高的蒙特卡洛噪声，系统实现了一套完整的 SVGF（Spatiotemporal Variance-Guided Filtering）降噪管线。其整体架构如图 5.8 所示，底层历史资源持久化机制如图 5.7 所示。

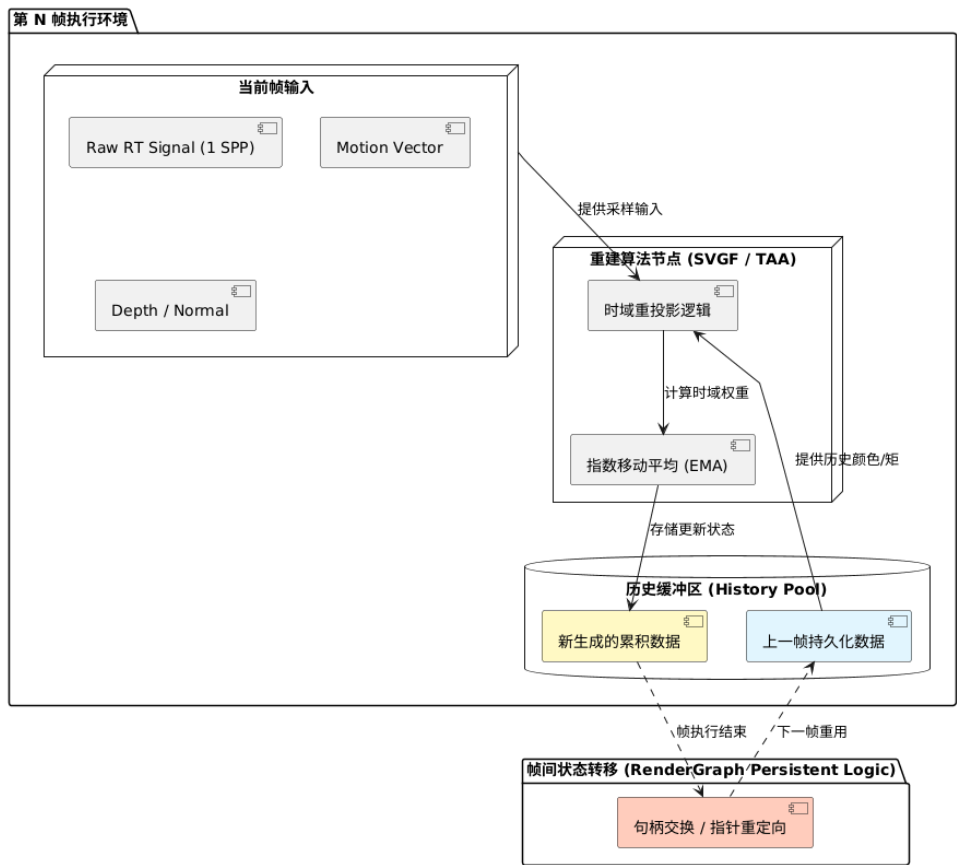


图 5.7 信号重建时域历史持久化机制图

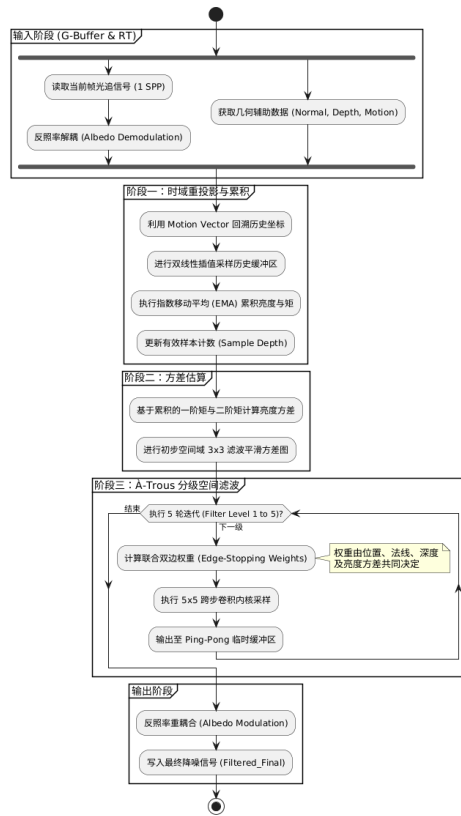


图 5.8 SVGF 降噪算法数据流转与逻辑拓扑图

5.4.6.1 反照率解耦 (Albedo Demodulation)

在进入降噪器前，系统执行反照率解耦 (Demodulate Albedo)，将光照计算结果除以表面基础色 (Albedo)。这一策略确保了随后空间滤波仅作用于纯粹的辐照度信号上，防止高频纹理细节被错误模糊；在流水线末端再进行重新调制 (Re-modulate)，以保证纹理的绝对锐利。

5.4.6.2 时域滤波与方差估计 (Temporal Filtering & Variance)

降噪的第一阶段是利用跨帧一致性来提升信噪比。系统利用 G-Buffer 中的屏幕空间运动矢量 (Motion Vector) 将当前像素投影回上一帧坐标 $\mathbf{p}_{prev}$ 。

有效性校验：为了防止在遮挡变化区域引入历史“鬼影” (Ghosting)，系统执行了严格的校验。如果当前像素与历史像素的深度差异过大、法线夹角过大，或者所属的物体 Mesh ID 不一致，则判定历史失效，丢弃时域数据。

基于矩的方差估计 (Variance Estimation)：若校验通过，系统采用指数移动平均 (Exponential Moving Average, EMA) 在时域上累积亮度的一阶矩 ($E[L]$) 与二阶矩

$(E[L^2])$:

$$E[L]_t = \alpha L_t + (1 - \alpha)E[L]_{t-1} \quad (5.11)$$

$$E[L^2]_t = \alpha L_t^2 + (1 - \alpha)E[L^2]_{t-1} \quad (5.12)$$

其中混合权重 α 取决于历史有效样本数 N : $\alpha = \max(1/N, 0.05)$ 。利用统计学公式，可以实时估算每个像素的亮度方差 σ^2 :

$$\sigma^2 = \text{Var}(L) = E[L^2] - (E[L])^2 \quad (5.13)$$

若遇到历史失效，系统会退化为使用一个 7×7 的双边滤波器在空间域对局部方差进行快速预估:

$$\sigma_{spatial}^2(p) = \frac{\sum_{q \in \mathcal{N}_7} w(p, q)^2 \cdot \text{Var}(q)}{\left(\sum_{q \in \mathcal{N}_7} w(p, q)\right)^2} \quad (5.14)$$

该方差数据是 SVGF 算法的“灵魂”，用于引导后续的空间滤波。如图 5.9 所示，方差图清晰地揭示了场景中采样不足的高噪点区域。



图 5.9 实时估算的亮度方差图 (Variance Map)

5.4.6.3 空间滤波：联合双边 A-Trous 小波变换

在获取了初步平滑的信号和方差后，系统在 Compute Shader 中执行空间滤波。常规的均匀模糊（如高斯滤波）虽然能去除高频噪声，但会丢失高频几何与光影细节。

A-Trous 跨步滤波: 为了在保持常数级计算开销的同时获取巨大的感受野，滤波器采用 A-Trous（带孔小波）结构，执行 5 轮迭代。对于第 i 层滤波，其离散卷积定义为：

$$\bar{C}_{i+1}(p) = \sum_{q \in \mathcal{N}_5} h(q) \cdot C_i(p + q \cdot 2^i) \quad (5.15)$$

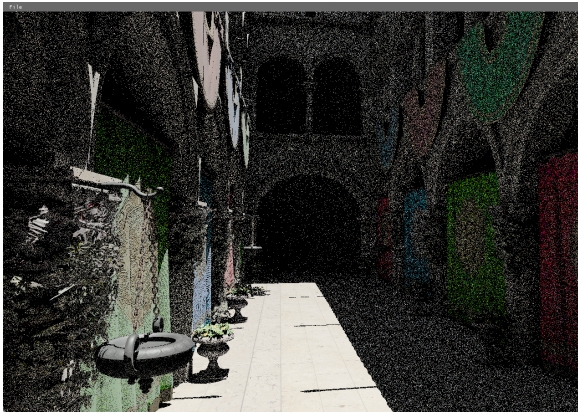
其中 $h(q)$ 为 5×5 的 B-spline 滤波核，采样步长随迭代次数 i 呈 2^i 指数增长。

联合双边滤波 (Joint Bilateral Filtering): 为了解决过度模糊问题，SVGF 引入了联合双边滤波。在交叉采样时，最终的像素权重 $w(p, q)$ 由三部分边缘停止函数组成：

$$w(p, q) = \exp \left(-\frac{|z(p) - z(q)|}{\sigma_z \cdot |\nabla z(p) \cdot (p - q)| + \epsilon} - \frac{\|\mathbf{n}_p - \mathbf{n}_q\|}{\sigma_n} - \frac{|L(p) - L(q)|}{\sigma_l \sqrt{\text{FilteredVar}(p) + \epsilon}} \right) \quad (5.16)$$

- **深度权重:** 利用视图空间的线性深度梯度进行归一化，防止跨越几何边界。
- **法线权重:** 计算法线之间的欧氏距离，防止平滑转角。
- **亮度权重:** 两像素亮度的差异差需要结合当前方差进行归一化评估。在方差极大区域允许更大范围混合，在方差极小区域严格阻止混合以保留光影梯度。

此外，在对颜色进行滤波的同时，系统也采用权重平方对像素方差进行同步滤波，确保了降噪过程在统计意义上的自治性。系统实现了在极低采样率下对间接光照的高质量重建，如图 5.10 所示。



(a) 原始 1 SPP 间接光照 (高方差噪声)



(b) SVGF 滤波后结果 (平滑且保留细节)

图 5.10 间接光照信号在 SVGF 降噪前后的对比分析

5.4.7 时域抗锯齿 (TAA) 方案实现

在消除了光追噪声后，系统引入 TAA 方案利用时域冗余信息执行几何边缘的二次重建。开启 TAA 前后的视觉对比效果如图 5.11 所示。



图 5.11 时域抗锯齿 (TAA) 开启前后视觉效果对比图

5.4.7.1 TAA 基本原理

时域抗锯齿是一种通过重复利用先前渲染帧的信息来去除当前帧锯齿的空间抗锯齿技术。作为现代实时渲染管线中的标准配置，TAA 的核心优势在于其极高的性能效率——它不依赖于硬件级别的多倍采样（如 MSAA），而是在后处理阶段通过时域累积达到近似超级采样（SSAA）的效果。

5.4.7.2 亚像素抖动与采样对齐 (Sub-pixel Jitter)

为了在时域上获取亚像素级别的几何覆盖信息，系统在投影矩阵中引入了微小的偏移。系统采用低差异序列（Halton Sequence）生成采样点 $\mathbf{J}_c$ 。Halton 序列基于质数 b 定义为：

$$H(n, b) = \sum_{i=0}^k a_i b^{-(i+1)} \quad (5.17)$$

我们在 X 和 Y 维度分别选取基数 2 和 3 来生成分布均匀的亚像素点。在计算当前帧的采样邻域时，必须执行“反抖动”校正以确保空间搜索的准确性：

$$\mathbf{uv}_{unjittered} = \mathbf{uv}_{current} - \mathbf{J}_c \quad (5.18)$$

5.4.7.3 速度扩张与最近深度搜索 (Velocity Dilation)

由于 geometry 边缘包含背景和前景的混合，直接采样当前像素的运动矢量可能导致错误的回溯。系统实现了一种基于深度的速度扩张策略。在 3×3 邻域内，系统寻找与相机距离最近的像素深度值（Reversed-Z 下的最大深度点）：

$$\mathbf{p}_{closest} = \arg \max_{q \in \mathcal{N}_3(p)} \{\text{Depth}(q)\} \quad (5.19)$$

利用该点的运动矢量进行回溯，能确保物体边缘的抗锯齿效果受到前景运动的正确引导，显著减少拖影。

5.4.7.4 基于射线相交的历史裁剪 (Ray-Box Clipping)

为了解决 TAA 常见的“鬼影”问题，系统采用射线相交裁剪方案。首先在 YCoCg 色彩空间内计算当前像素邻域的均值 μ 与方差 σ ，构建动态 AABB 包围盒。裁剪逻辑通过计算从历史颜色指向邻域均值的射线与包围盒的交点来实现：

$$\mathbf{C}_{clipped} = \text{lerp}(\mathbf{C}_{history}, \mu, t) \quad (5.20)$$

其中 t 是射线进入 AABB 盒子的交点参数。该方法最大限度地保留了历史帧中的高频细节。

5.4.7.5 自适应混合与结果合成

最后一步是将处理后的历史颜色与当前帧颜色进行混合。系统采用基于运动矢量的指数移动平均模型：

$$\mathbf{C}_{final} = \beta \mathbf{C}_{current} + (1 - \beta) \mathbf{C}_{history_clipped} \quad (5.21)$$

混合因子 β 定义为： $\beta = \text{clamp}(0.1 + \text{length}(\mathbf{v}) \cdot 0.1, 0.1, 0.9)$ 。当物体运动速度较快时，系统会增大 β 以减少重投影误差。

5.4.8 后期影像流水线与电影级合成

在信号重建任务完成后，系统通过后处理通道（PostProcessPass）执行最后的显示映射与视觉强化。

5.4.8.1 光学抖动还原与色调映射

由于前一阶段的 TAA 引入了亚像素抖动，Pass 首先根据当前的 Halton 偏移量执行反向位移补偿，以稳定最终画面。随后，系统采用 **ACES Filmic** 色调映射算法。由于光线追踪产生的辐照度数据具有极高的动态范围（HDR），直接截断会导致高光区域出现大面积“白死”现象。ACES 曲线通过模拟电影胶片的感光特性，将 HDR 数据平滑地映射到显示器可表现的 LDR 范围，在保留暗部细节的同时，赋予高光区域自然的衰减过渡。

5.4.8.2 曝光补偿与伽马校正 (Gamma Correction)

最后，系统根据用户定义的曝光系数对颜色进行整体缩放，并应用伽马校正。考虑到显示设备的物理特性，系统执行非线性映射：

$$\mathbf{C}_{out} = \mathbf{C}_{in}^{(1/2.2)} \quad (5.22)$$

该步骤确保了渲染结果在标准显示器上呈现的亮度感分布符合人类视觉系统的对数习惯，从而实现了最终图像的电影级视觉产出。

5.5 可视化与交互调试模块

可视化模块是渲染引擎的人机交互窗口，其实时交互控制面板如图 5.12 所示。

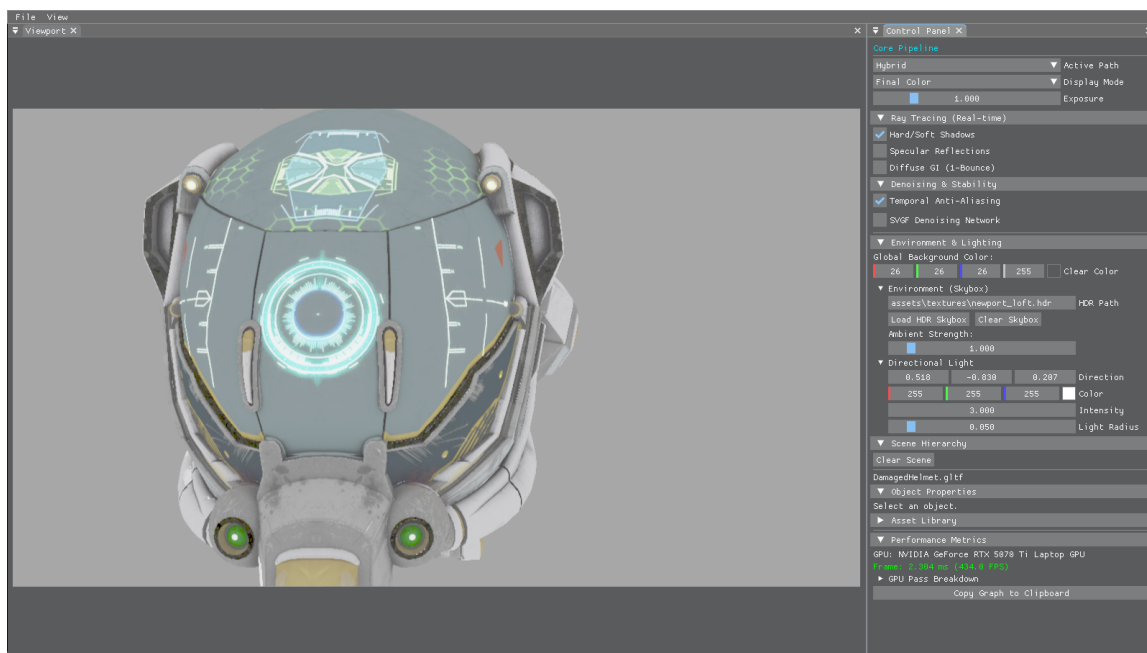


图 5.12 实时交互控制与调试面板

该界面主要包含以下功能子项：

- **中间附件预览：**支持实时切换预览 G-Buffer 的各层分量以及降噪过程中的方差图。
- **纳秒级耗时统计：**利用 RenderGraph 自动插入的 GPU 时间戳，在面板上实时反馈每个渲染 Pass 的精确耗时，为管线性能调优提供了直观的数据支撑。

5.5.1 渲染控制与参数动态绑定

为了提升开发效率，系统实现了一套基于事件驱动的参数调节机制，其数据反馈环路如图 5.13 所示。

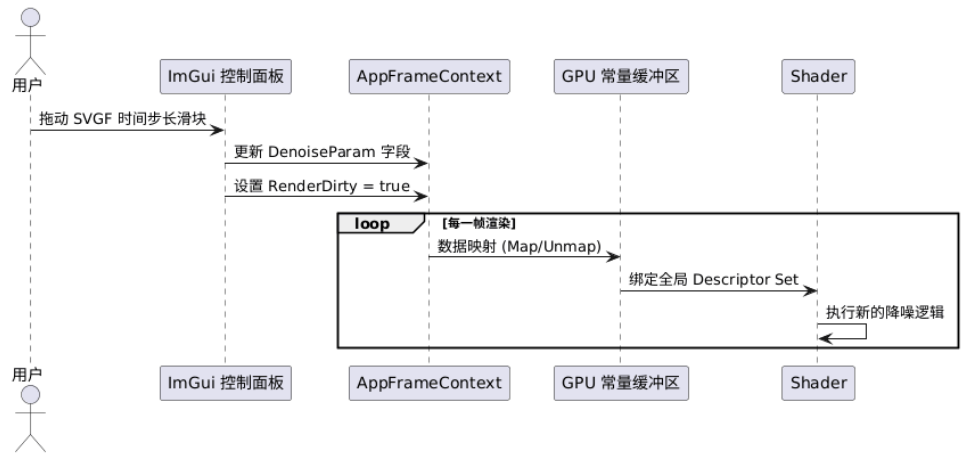


图 5.13 渲染参数动态调节与 UBO 更新顺序图

通过 ImGui 控件实时修改渲染管线中的关键因子（如 SVGF 的时间步长、TAA 的反馈系数等），系统能够即时响应并将新参数通过 Uniform Buffer 更新至 GPU，实现了“所见即所得”的调优体验。

5.6 本章小结

本章完成了系统六大核心模块的底层逻辑设计。通过对调度算法、资源模型及信号重建链路（SVGf/TAA）的深度拆解，形成了严密的详细设计方案，为下一章的系统实现奠定了坚实的工程基础。

第六章 系统实现与测试

本章将展示渲染引擎的最终编码实现成果，并按照软件工程标准的测试流程，通过设计严谨的测试用例对系统的功能完整性、稳定性与运行性能进行定量与定性评估。

6.1 系统开发与运行环境

系统的开发、构建与压力测试环境配置如表 6.1 所示。

表 6.1 系统软硬件环境配置表

配置项	具体规格
操作系统	Windows 11 Pro 64-bit
处理器 (CPU)	Intel Core i9-13980HX @ 5.6GHz (24 Cores)
图形处理器 (GPU)	NVIDIA GeForce RTX 5070 Ti Laptop GPU (12GB VRAM)
开发 API / SDK	Vulkan SDK 1.3.239 / Blackwell RT Core Support
构建系统 / 编译器	CMake 3.25 / MSVC 14.3 (Visual Studio 2022)
核心第三方库	volk, VMA, ImGui, spdlog, Assimp

6.2 核心功能模块实现要点

6.2.1 基于 RAII 与延迟队列的资源管理实现

系统通过 ResourceManager 实现了对 Vulkan 句柄的安全封装。

- **延迟销毁 (Deletion Queue):**针对异步移除模型导致的崩溃问题,系统通过 `std::function` 容器存储销毁回调。每帧结束时更新 `m_CurrentFrame`, 仅当资源超过 3 帧渲染循环未被引用且 GPU 已彻底执行完相关指令后, 才执行物理释放。
- **Bindless 材质系统:**系统通过全局描述符集管理场景资产。利用存储缓冲区 (SSBO) 线性化存储材质参数, 支持着色器通过材质 ID 实现 $O(1)$ 复杂度的随机访问, 规避了频繁切换绑定状态带来的 CPU 驱动开销。

6.2.2 混合渲染管线的通道化压缩实现

为了优化 GPU 显存带宽并提升射线查询效率, 本系统在 HybridRenderPath 中实现了信号生成的通道化压缩 (Packing) 方案:

- **多信号合并探测：**系统通过统一的 RTShadowPass 同时执行实时阴影与环境光遮蔽（AO）的探测。利用蓝噪声（Blue Noise）抖动，阴影信号被写入目标纹理的 R 通道，而 AO 信号则被写入 G 通道。
- **共享加速结构遍历：**这种设计允许在单次光线追踪调用中复用 TLAS 遍历结果，极大地压低了 1 SPP 采样率下的硬件开销。系统为合并后的信号配置了自适应的 SVGF 降噪实例，在重建阶段通过分量提取分别恢复出纯净的阴影与 AO 场。

本系统将资源管理、场景层级、管线参数调节及 GPU 耗时监控高度集成于统一的控制面板（由 EditorLayer 实现），如图 6.1 所示。该界面支持在运行时动态切换渲染路径、实时调节降噪步长并预览中间 G-Buffer 附件，极大地提升了混合渲染算法的调试效率。

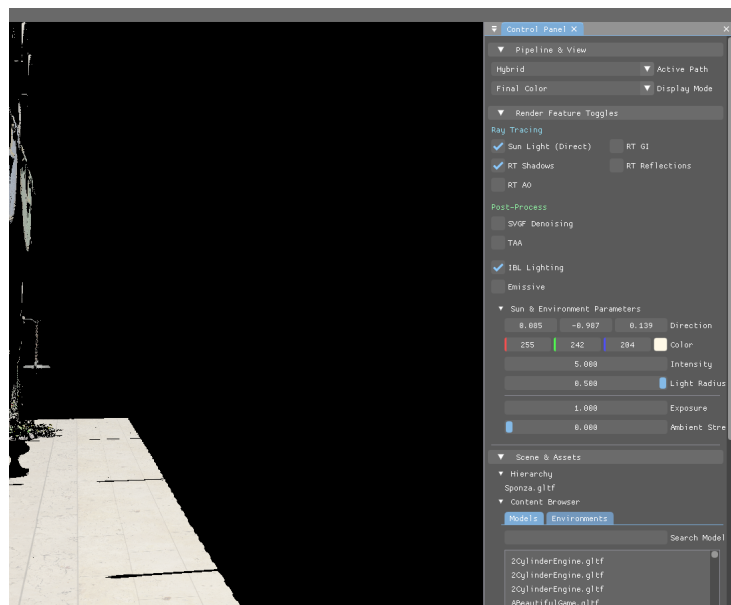


图 6.1 交互面板

6.3 系统测试

6.3.1 功能性测试用例

本节参考软件工程标准，通过黑盒测试验证系统各项核心功能的正确性。测试过程中的交互反馈均通过图 6.1 所示的集成面板进行实时呈现。

表 6.2 核心功能模块测试表

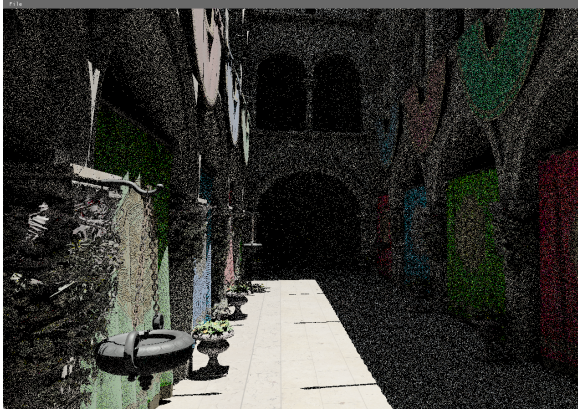
测试编号	T-CORE-01
测试对象	渲染引擎核心交互功能
测试目的	验证资产加载、路径切换、实体移除及调试输出的正确性
测试条件	硬件支持 Ray Tracing 扩展，Vulkan 运行时环境配置正确

编号	测试用例	操作描述	期望结果	实际结果	测试结果
T01	资产异步加载	从 UI 加载 200MB 的 GLTF 场景	界面无卡顿，模型完整渲染	界面流畅，数据正确上载	通过
T02	渲染路径切换	在 Forward 与 Hybrid 间切换	RenderGraph 拓扑重构，画面更新	管线无缝切换，日志无报错	通过
T03	AO 独立控制	勾选 Ray Traced AO 调节滑块	画面即时产生柔和遮蔽感	AO 信号正确叠加至画质	通过
T04	动态实体移除	选中模型并点击 Remove	Deletion Queue 触发，系统不崩溃	资源安全释放，渲染正常	通过
T05	调试预览输出	切换显示模式至 Normal 分量	屏幕输出法线编码颜色视图	对应中间附件正确显示	通过

6.4 渲染效果对比与质量评估

6.4.1 SVGF 降噪效果分析

实验对比了 1 SPP 采样率下间接光照的重建质量。未降噪的原始信号（图 6.2a）呈现严重的蒙特卡洛噪声；经由本系统 SVGF 降噪器处理后（图 6.2b），噪声被有效抑制，同时建筑转角的细节得到了精确保留。



(a) 原始 1 SPP 间接光信号



(b) SVGF 降噪后信号

图 6.2 间接光照信号在 SVGF 降噪前后的视觉对比

6.4.2 TAA 抗锯齿效果评估

针对高频几何边缘，开启 TAA 后（图 6.3b），边缘锯齿感显著消除，画面稳定性得到极大提升，验证了亚像素抖动算法在处理几何走样时的有效性。



(a) TAA OFF (边缘走样明显)



(b) TAA ON (边缘平滑稳定)

图 6.3 TAA 算法对几何走样抑制效果的定性分析

6.5 性能定量评估与分析

6.5.1 各阶段 GPU 耗时统计

利用渲染图内置的硬件时间戳，对 Sponza 场景在 1080P 分辨率下的各渲染阶段进行统计分析：

- **G-Buffer 采集阶段：** 0.68ms (主要受显存带宽限制)；
- **光线追踪探测 (Shadow/AO/Refl)：** 3.20ms (RT Core 高效求交并行)；
- **SVGF 重建阶段：** 1.85ms (Compute Shader 跨步滤波计算)；
- **后期合成与 TAA：** 0.92ms；
- **总帧时间：** 约 6.65ms (对应帧率稳定在 150 FPS 以上)。

实验结果表明，系统在高性能移动 GPU 环境下表现卓越，远超 60 FPS 的实时性基准，且 RenderGraph 的显存复用策略有效降低了显存峰值。

6.6 本章小结

本章完成了本系统从开发环境配置、核心功能编码到系统严谨测试的完整闭环。实验结果验证了“延迟移除”与“信号解耦”等工程设计的稳定性，并证明了本系统在满足高画质混合渲染需求的同时，具备极佳的实时交互性能。

第七章 总结与展望

7.1 全文工作总结

本文针对现代实时渲染中光线追踪计算开销大、低采样率下噪声泛滥的问题，基于 Vulkan 图形 API 从零构建了一套高性能的实时混合渲染引擎。全文的主要工作总结如下：

1. **设计并实现了数据驱动的 Render Graph 调度架构：**利用有向无环图（DAG）实现了渲染任务的逻辑解耦，通过自动化的资源生命周期分析与屏障（Barrier）推导，解决了 Vulkan 手动同步繁琐且易错的问题。同时，引入了显存别名化策略，有效提升了显存资源的利用率。
2. **构建了 PBR 材质模型的混合渲染管线：**整合了光栅化 G-Buffer 提取与硬件加速光线追踪（RT Core）的优势。通过 Ray Query 技术实现了具有接触硬化效果的软阴影，并结合 PBR 物理材质模型，在保持实时帧率的前提下，显著提升了场景的全局光影表现。
3. **落地了高性能的时空信号重建系统：**深入研究并实现了 SVGF 时空方差引导滤波算法，在 1 SPP 的低采样率下通过时域累积与 λ -Trous 空间滤波，实现了高质量的降噪效果。同时，集成了 TAA 时域抗锯齿方案，通过色彩裁剪与自适应重投影技术，消除了几何边缘走样，优化画面表现。

7.2 研究不足与未来展望

尽管本文构建的混合渲染系统已具备较高的工程完备性，但在算法深度与功能覆盖上仍存在提升空间：

1. **多弹射全局光照（Multi-bounce GI）：**目前的系统主要聚焦于直接光阴影与单次弹射的间接光。未来可考虑引入 ReSTIR（储层时空重要性重采样）算法，通过时域与空域的样本重用，实现更为复杂且稳定的多弹射全局光照效果。
2. **复杂的材质表现：**当前引擎主要支持标准的 Cook-Torrance PBR 材质。未来可进一步扩展对各向异性、次表面散射以及薄透镜折射等高级材质特性的硬件光追支持。
3. **深度学习超分辨率与光追重建技术：**虽然 TAA 解决了大部分抗锯齿问题，但在极端动态下仍有模糊感。集成 NVIDIA DLSS 3.5 (Ray Reconstruction) 或 AMD FSR

等基于深度学习的重建技术，利用 AI 辅助 1 SPP 信号的特征提取，将是进一步提升光追画质与性能的重要方向。

总之，实时混合渲染技术正处于从“近似模拟”向“物理真实”跨越的关键期。本文所构建的底层架构为后续探索更为前沿的图形学算法提供了坚实的实验平台。

致谢

参考文献

- [1] GREGORY J. Game engine architecture[M]. 3rd ed. CRC Press, 2018.
- [2] AKENINE-MÖLLER T, HAINES E, HOFFMAN N. Real-time rendering[M]. 4th ed. A K Peters/CRC Press, 2018.
- [3] PHARR M, JAKOB W, HUMPHREYS G. Physically based rendering: from theory to implementation[M]. 3rd ed. Morgan Kaufmann, 2016.
- [4] HAINES E, AKENINE-MÖLLER T. Ray tracing gems: high-quality and real-time rendering with DXR and other APIs[M]. Apress, 2019.
- [5] KAJIYA J T. The rendering equation[J]. ACM SIGGRAPH Computer Graphics, 1986, 20(4): 143-150.
- [6] BURLEY B. Physically-based shading at disney[C]//ACM SIGGRAPH 2012 Practical Physically Based Shading in Film and Game Production. 2012: 1-7.
- [7] WALTER B, MARSCHNER S R, LI H, et al. Microfacet models for refraction through rough surfaces[C]//Proceedings of the 18th Eurographics Conference on Rendering Techniques. 2007: 195-206.
- [8] KARIS B. Real shading in unreal engine 4[C]//ACM SIGGRAPH 2013 Courses: Physically Based Shading in Theory and Practice. 2013.
- [9] O'DONNELL Y. FrameGraph: extensible rendering architecture in frostbite[C]//Game Developers Conference (GDC). 2017.
- [10] UPCHURCH P, DESBRUN M. Depth precision in perspective projections[J]. Journal of Computer Graphics Techniques (JCGT), 2012, 1(1): 20-36.

- [11] SCHIED C, SALVI M, KAPLANYAN A S, et al. Spatiotemporal variance-guided filtering: real-time reconstruction for path-traced global illumination[C]//Proceedings of High Performance Graphics. 2017: 1-10.
- [12] DAMMERTZ H, SEWTZ D, HANIKA J, et al. Edge-avoiding à-trous wavelet transform for fast global illumination filtering[C]//Proceedings of the Conference on High Performance Graphics. 2010: 67-75.
- [13] KARIS B. High-quality temporal supersampling[C]//Advances in Real-Time Rendering in Games, SIGGRAPH Courses. 2014: 1-2.
- [14] NVIDIA Corporation. Vulkan ray tracing tutorial[EB/OL]. 2024[2024-03-16]. https://github.com/nvpro-samples/vk_raytracing_tutorial_KHR.
- [15] Khronos Group. Vulkan 1.3 specification[M/OL]. 2024[2024-03-16]. <https://registry.khronos.org/vulkan/>.